

**POSTS AND TELECOMMUNICATIONS INSTITUTE OF
TECHNOLOGY**
Faculty of Information Technology

Intelligent System Development
ASSIGNMENT 05

**Convolutional Neural Networks:
CNN-3 vs. CNN-5 in TensorFlow/Keras and PyTorch**

Instructor / Lecturer: Assoc. Prof. Dinh Que Tran, Ph.D.
Email: quetd@ptit.edu.vn, tdque@yahoo.com

Student Name: Nguyễn Văn Trường
Student ID: B23DCCE095
Semester: I.2025 – 2026
Class: E23CNPM02
Submission: Individual Project

Hanoi, 2026

Contents

1	Executive Summary	2
2	CNN Fundamentals	2
2.1	Convolution	3
2.2	ReLU, pooling, batch normalization	3
2.3	Output, loss and training	3
2.4	One image through CNN-3, step by step	3
2.5	Learning: one backpropagation step by hand	5
2.6	One epoch of training	6
2.7	Training mode vs. evaluation mode	6
2.8	1-D convolution on a table	6
2.9	Metrics	7
3	Project Structure and Setup	7
4	Datasets	7
4.1	SVHN	8
4.2	GTSRB	9
4.3	Diabetes (Playground Series S5E12)	9
5	Models	10
5.1	TensorFlow, Keras and PyTorch	11
5.2	Implementation (TensorFlow/Keras)	12
5.3	Implementation (PyTorch)	12
6	Results per Dataset	13
6.1	SVHN	14
6.2	GTSRB	16
6.3	Diabetes	18
7	Cross-Dataset Comparison	20
7.1	Keras vs. PyTorch outputs	21
7.2	Why depth helps on images: the receptive field	25
7.3	Why a CNN does not help on the diabetes table	26
8	Conclusion and Limitations	27
A	Appendix	27
A.1	Additional figures	27
A.2	How to run the project	30

1 Executive Summary

This project sets up convolutional networks of **3** and **5** conv layers (CNN-3, CNN-5) using both **TensorFlow/Keras** (TF) and **PyTorch** (PT), then fits them to three datasets: **SVHN** (99,289 house-number photos, 10 classes), **GTSRB** (51,839 traffic-sign photos, 43 classes) and a **diabetes** table (300,000 patients, 24 features, binary target). Every matched pair of runs shares the split, architecture, parameter count, optimizer, batch size, early-stopping rule and seed, so the only things that change are the depth of the network and the framework used. Each trained model is tagged with a run ID of the form <dataset>-<framework>-<model>, for instance SV-PT-CNN-5.

Table 1: All 12 runs (test set). s/epoch = median training time per epoch, CPU.

Dataset	Run ID	Accuracy	Macro F1	Epochs	s/epoch	Train time (s)	Params
SVHN	SV-TF-CNN-3	0.9012	0.8959	14	19.8	279	129,290
	SV-TF-CNN-5	0.9398	0.9363	20	42.9	865	175,658
	SV-PT-CNN-3	0.9091	0.9069	20	28.1	557	129,290
	SV-PT-CNN-5	0.9319	0.9282	12	61.5	750	175,658
GTSRB	GT-TF-CNN-3	0.9801	0.9745	20	10.2	207	137,771
	GT-TF-CNN-5	0.9924	0.9893	12	21.9	268	184,139
	GT-PT-CNN-3	0.9685	0.9621	14	14.2	199	137,771
	GT-PT-CNN-5	0.9973	0.9979	19	30.0	572	184,139
Diabetes	DB-TF-CNN-3	0.6452	0.6304	7	9.7	70	191,169
	DB-TF-CNN-5	0.6358	0.6275	5	18.6	96	391,329
	DB-PT-CNN-3	0.6400	0.6288	8	21.1	167	191,169
	DB-PT-CNN-5	0.6454	0.6303	6	38.4	230	391,329

Key takeaways

1. On the two image datasets, adding depth helps: CNN-5 beats CNN-3 under both frameworks, with macro F1 up by +0.021 to +0.040 on SVHN and +0.015 to +0.036 on GTSRB. The best model overall is GT-PT-CNN-5, with a macro F1 of 0.9979.
2. The diabetes dataset tells a different story. All four CNN variants sit in a narrow band of macro F1 (0.627–0.630) and ROC-AUC (0.694–0.696), and a much smaller MLP (6,977 parameters versus 391,329) reaches ROC-AUC 0.695 as well. The features themselves carry a weak signal, so no amount of extra depth changes that.
3. Framework choice has a modest, dataset-dependent effect: TF and PT macro F1 stay within 0.013 of each other on every dataset, though which framework comes out ahead flips between CNN-3 and CNN-5, and their parameter counts are identical either way (checked directly in code).
4. The cost of depth is time, not accuracy loss: CNN-5 needs 1.8–2.2× as long per epoch as CNN-3. On this CPU-only run, PyTorch was 37–43% slower per epoch than Keras on the two image datasets and roughly 2× slower on the tabular one.

2 CNN Fundamentals

Each term below is defined a single time and keeps that meaning throughout the rest of the report. Notation used: N samples; input $C \times H \times W$ (channels, height, width); kernel size K , stride S , padding P ; F filters (output channels); logits z , predicted probability p , true label y .

2.1 Convolution

A convolutional layer passes F learnable **filters**, each of size $K \times K \times C_{in}$, across the input. At every position it produces a single value belonging to a **feature map**:

$$\text{out}(i, j) = \sum_{u, v, c} \text{window}(i+u, j+v, c) \cdot w(u, v, c) + b, \quad H_{out} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1.$$

Worked example (SVHN notebook, §4.2). Take a 5×5 patch that contains a vertical stroke (columns 0 0 1 1 0) and a vertical-edge filter with rows $[-1, 0, 1]$: at window $[0, 0, 1]$ the result is $(-1 \cdot 0 + 0 \cdot 0 + 1 \cdot 1) \times 3 = 3$ (the stroke’s left edge), while window $[1, 1, 0]$ yields -3 (its right edge). This value is cross-checked against the framework’s own convolution output in the notebook. Setting $K = 3$, $S = 1$ and “**same**” padding ($P = 1$) preserves the spatial size, $32 \times 32 \rightarrow 32 \times 32$.

2.2 ReLU, pooling, batch normalization

Table 2: Layer operations with worked examples (from the notebooks).

Layer	Computation	Example
ReLU	$\max(0, x)$	$-3 \rightarrow 0, \quad 3 \rightarrow 3, \quad -1.3 \rightarrow 0$
Max pooling 2×2	largest value of each 2×2 block; halves H, W ; no parameters	$[[0.1, 0.9], [0.4, 0.2]] \rightarrow 0.9$
Batch normalization	$\hat{x} = (x - \mu) / \sqrt{\sigma^2 + \epsilon}$, out = $\gamma \hat{x} + \beta$, per channel over the batch	$[1, 3, 5] \rightarrow [-1.22, 0, 1.22]$; dark/bright pixels $[0.05, 0.10, 0.80, 0.90] \rightarrow [-1.06, -0.93, 0.87, 1.12]$
Global average pooling	mean of each feature map	$4 \times 4 \times 128 \rightarrow 128$
Dropout (0.3)	in training, 30% of values set to 0	reduces overfitting

The **receptive field** is the region of the input that can affect a single output unit. It expands by $RF \leftarrow RF + (K - 1) \cdot j$ at each layer, with the jump j doubling every time pooling is applied, reaching 22×22 pixels for CNN-3 and 28×28 for CNN-5 (out of a 32×32 image).

Parameter counts. A conv layer contributes $(K \cdot K \cdot C_{in} + 1) \cdot F$, e.g. the first layer gives $(3 \cdot 3 \cdot 3 + 1) \cdot 32 = 896$; BatchNorm adds $2F$ trainable values (γ, β) ; a Dense layer adds $(\text{inputs} + 1) \cdot \text{units}$.

2.3 Output, loss and training

- **Softmax** (multi-class): $p_k = e^{z_k} / \sum_j e^{z_j}$. **Cross-entropy**: $L = -\ln p_{\text{true}}$. For instance, with logits “3” = 2, “8” = 1 and everything else 0, $p(\text{“3”}) = 0.41$, giving a loss of 0.90 if the true image is a 3 and 1.90 if it is actually an 8. Before any training the model starts near $\ln 10 = 2.30$ (SVHN) or $\ln 43 = 3.76$ (GTSRB).
- **Sigmoid** (binary): $p = 1 / (1 + e^{-z})$. **Binary cross-entropy**: $L = -[y \ln p + (1 - y) \ln(1 - p)]$. For example, $z = 1.2 \Rightarrow p = 0.77$, so the loss is 0.26 when $y = 1$ and 1.47 when $y = 0$.
- **Adam**: $w \leftarrow w - \eta \hat{m} / (\sqrt{\hat{v}} + \epsilon)$, $\eta = 0.001$. An **epoch** is one full sweep through the training batches (256 samples each). **Early stopping** halts training once 3 epochs pass without a new lowest validation loss, and the weights from the best epoch are restored.

2.4 One image through CNN-3, step by step

Figures 1 and 2 trace a single SVHN test image (a “3”) as it passes through a PyTorch CNN-3. These figures come from a CNN-3 trained for only 3 epochs, purely to demonstrate the flow of information; the actual reported results are taken from the complete runs described in Section 6.

Table 3: What happens to one image in CNN-3 (SVHN).

Step	Output shape	What it means
input	$3 \times 32 \times 32$	3,072 values in $[0, 1]$: the red, green and blue intensity of every pixel
block 1	$32 \times 16 \times 16$	32 filters each hunt for one small pattern (an edge, a colour change); each resulting map marks <i>where</i> that pattern turned up, and pooling shrinks the size by half
block 2	$64 \times 8 \times 8$	the filters combine block-1 maps into curves and stroke fragments spanning a wider area
block 3	$128 \times 4 \times 4$	each unit now covers 22×22 pixels, enough to capture parts of a digit (a loop, a bar)
GAP	128	averaging each 4×4 map answers “how strongly is pattern i present anywhere”
Dense 256 $\rightarrow$ 10		weighted combinations of the 128 values produce one score (logit) per digit
Dense 10		
softmax	10	the scores are converted into probabilities that add up to 1

One SVHN image through CNN-3: 4 strongest feature maps after each conv block

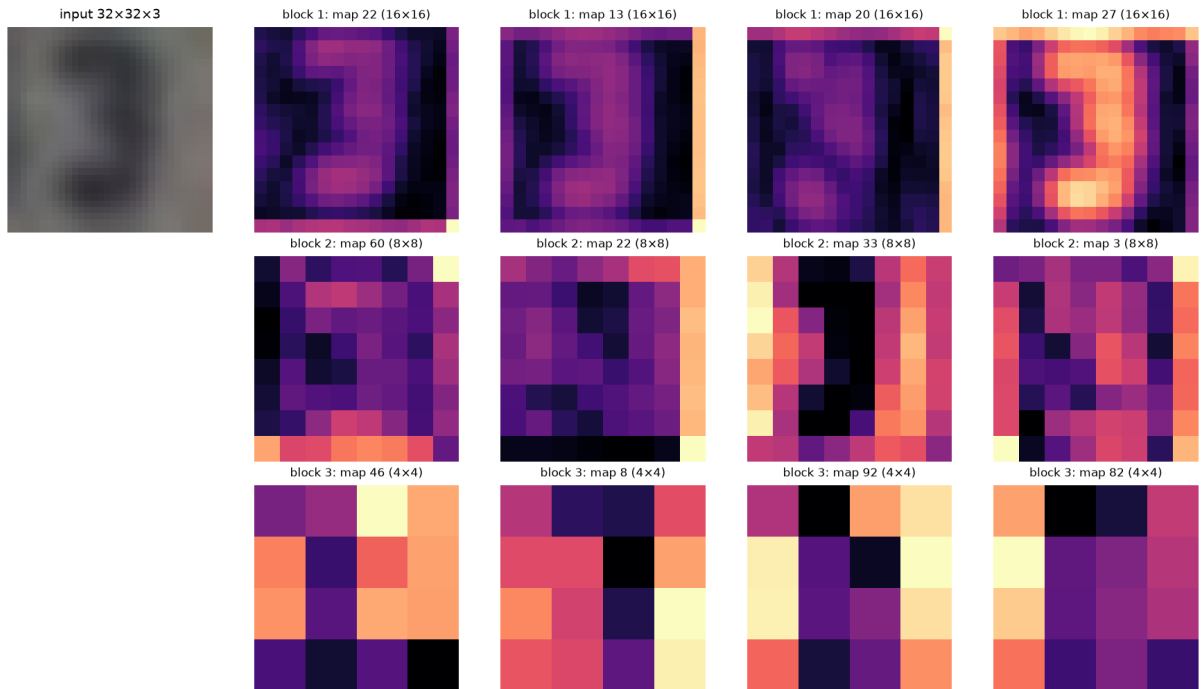


Figure 1: Input image alongside the 4 strongest feature maps after each conv block (bright = strong response). The maps shrink and grow more abstract at each stage: block 1 still resembles the digit, while block 3 only flags *where* pieces of the digit lie.

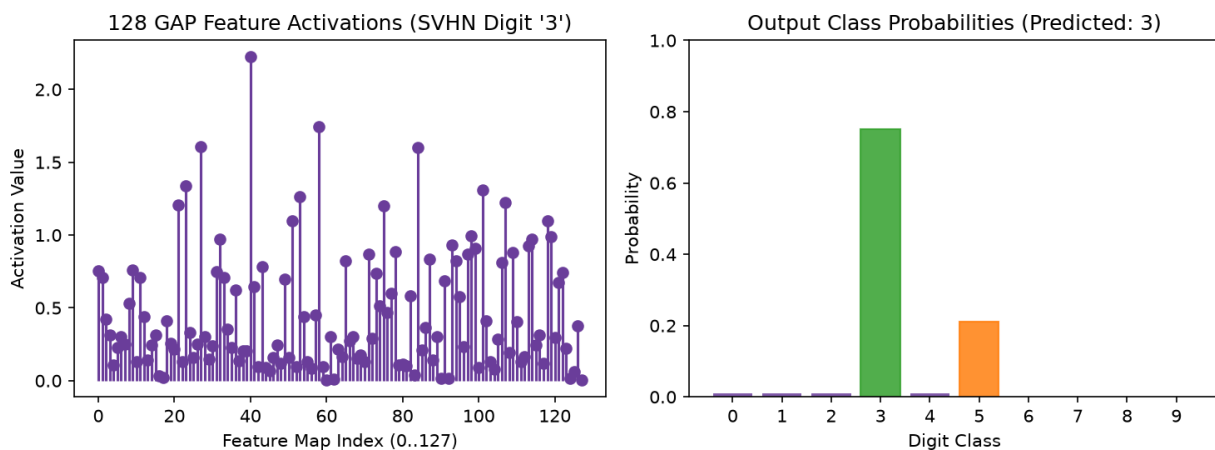


Figure 2: The 128 values produced by global average pooling (left) and the resulting probabilities (right): $p(3) = 0.75$, $p(5) = 0.21$. The runner-up class “5” shares the “3”s lower curve, and this 5/3 pair also turns up as one of the most common confusions in the full results.

2.5 Learning: one backpropagation step by hand

Training nudges every weight slightly so the loss decreases. In PyTorch this is driven by `loss.backward()` (which computes the gradients) followed by `optimizer.step()` (which applies the update). The simplest possible case is a single neuron taking one input:

$$z = wx + b, \quad p = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad L = -[y \ln p + (1 - y) \ln(1 - p)].$$

Forward pass with $x = 2$, $w = 0.5$, $b = 0$ and true label $y = 1$: $z = 1.0$, $p = \sigma(1.0) = 0.731$, $L = -\ln 0.731 = 0.313$.

Backward pass (chain rule). For the sigmoid plus binary cross-entropy pairing the derivative reduces neatly to $\partial L / \partial z = p - y$, giving

$$\frac{\partial L}{\partial z} = 0.731 - 1 = -0.269, \quad \frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial w} = -0.269 \times x = -0.538, \quad \frac{\partial L}{\partial b} = -0.269.$$

Weight update (gradient descent, learning rate $\eta = 0.1$): $w \leftarrow w - \eta \partial L / \partial w$:

$$w = 0.5 - 0.1 \times (-0.538) = 0.554, \quad b = 0 - 0.1 \times (-0.269) = 0.027.$$

Verification: plugging in the updated weights gives $z = 0.554 \cdot 2 + 0.027 = 1.135$, $p = 0.757$, $L = 0.279$ — the loss dropped from 0.313 to 0.279 and p shifted closer to the true label of 1. Since the gradient was negative, the weight increased.

Inside a CNN the identical chain rule propagates backward through every layer:

Layer	How the gradient passes through
Softmax + cross-entropy	$\partial L / \partial z_k = p_k - [k = y]$, e.g. with $p = (0.75, 0.21, \dots)$ for (“3”, “5”) and true label “3”: -0.25 for “3”, $+0.21$ for “5”
Dense	$\partial L / \partial w_{ij} = \partial L / \partial z_j \cdot x_i$ (same rule as the single neuron above)
ReLU	the gradient passes through unchanged ($\times 1$) where the input was > 0 , and is blocked ($\times 0$) where it was clipped to 0
Max pooling	the entire gradient flows to whichever input was the maximum; the other three positions receive 0
Convolution	$\partial L / \partial w$ is the sum over every position of (the gradient at that output times the input window there): each filter learns from all the locations it was applied to

Adam subsequently rescales each weight’s update using running averages of its gradients (Section 2.3), and this whole cycle repeats once per batch across an epoch.

2.6 One epoch of training

An **epoch** covers one full pass over the training set, processed in batches of 256 samples:

Step	Code (PyTorch)	What happens
1	<code>model.train()</code>	switches to training mode: Dropout active, BatchNorm relies on the current batch’s μ, σ
2	<code>for x, y in train_loader:</code>	batches are reshuffled into a fresh random order each epoch
3	<code>optimizer.zero_grad()</code>	wipes out gradients left over from the previous batch
4	<code>loss = criterion(model(x), y)</code>	runs the forward pass and computes the loss (Sections 2.4, 2.3)
5	<code>loss.backward()</code>	derives the loss gradient for every weight (Section 2.5)
6	<code>optimizer.step()</code>	Adam nudges every weight accordingly
7	<code>model.eval()</code> + validation	measures loss and accuracy on the validation set without altering weights
8	early stopping	saves the weights whenever validation loss hits a new low; stops after 3 epochs with no improvement

Updates per epoch: SVHN’s 69,502 training images yield 272 weight updates; GTSRB’s 36,287 give 142; diabetes’ 210,000 rows give 821. **Early-stopping example (SV-TF-CNN-3):** validation loss bottomed out at epoch 11, showed no improvement through epochs 12–14, so training halted after epoch 14 and epoch 11’s weights were restored.

2.7 Training mode vs. evaluation mode

Two layer types act differently depending on whether the model is training or predicting, which is exactly why the code toggles between `model.train()` and `model.eval()` (Keras handles this switch automatically inside `fit` and `predict`).

Layer	Training	Evaluation (validation, test)
BatchNorm	normalizes using the current batch’s μ, σ and refreshes the running averages: $\text{running} = 0.9 \cdot \text{running} + 0.1 \cdot \text{batch}$	relies on the stored running μ, σ instead, so a given image produces the same output no matter which batch it lands in
Dropout (0.3)	zeroes out a random 30% of values and scales the rest by $1/0.7$ (e.g. $[1.0, 2.0, 0.5] \rightarrow [1.43, 0, 0.71]$)	is inactive: every value passes through unchanged

Skipping `model.eval()` would leave test-time predictions random (from Dropout) and dependent on batch composition (from BatchNorm).

2.8 1-D convolution on a table

Each patient is represented as a vector of d standardized features, treated as a length- d sequence with a single channel. A Conv1D filter of size 3 blends three adjacent features: $\text{out}[i] = w_1x[i-1] + w_2x[i] + w_3x[i+1] + b$. For example, $x = [0.5, -1.2, 0.3, 2.0, -0.4, 0.0]$ with $w = [1, 0, -1]$ gives $\text{out} = [1.2, 0.2, -3.2, 0.7, 2.0, -0.4]$ (verified against the framework’s own Conv1D). Table columns, unlike pixels, carry no inherent ordering, so which features count as “neighbours” depends on an arbitrary column arrangement; for this reason the tabular models **flatten** the final feature map

rather than averaging it (so that position still identifies which feature it is), and an MLP baseline is trained for comparison.

2.9 Metrics

For each class: $\text{precision} = TP/(TP + FP)$, $\text{recall} = TP/(TP + FN)$, $F1 = 2PR/(P + R)$. **Macro F1** takes the unweighted average of per-class F1 scores. Accuracy can be deceptive when classes are imbalanced: in a toy test set of 95 “big class” images and 5 “rare class” images, always guessing the big class scores accuracy 0.95 but only macro F1 0.49. **ROC-AUC** (for binary problems) measures the chance that a randomly chosen positive sample receives a higher p than a randomly chosen negative one (0.5 corresponds to random guessing).

3 Project Structure and Setup

Table 4: Folders and naming convention.

Item	Content / rule
dataset/sv, gt, db	raw Kaggle files (not committed) + <code>splits.npz</code> (row indices of the split)
notebook/	<code>sv_svhn.ipynb</code> , <code>gt_gtsrb.ipynb</code> , <code>db_diabetes.ipynb</code> : one self-contained notebook per dataset
report/	this report, <code>images/</code> (figures extracted from the notebooks), <code>make_summary.py</code>
Dataset / framework / model	SV, GT, DB / TF, PT / CNN-3, CNN-5 (number of conv layers)
Run ID and figure name	<code><dataset>-<framework>-<model></code> ; figure <code><run_id>_<plot>.png</code>

Every notebook follows the same layout: setup, data, exploration, TensorFlow, PyTorch, comparison, conclusion. Each code cell is preceded by a brief explanatory cell covering the formula, a worked example, or the relevant shapes.

Rules for a fair comparison. A stratified 70/15/15 split is computed once and reused by both frameworks; batch size 256; Adam ($\eta = 10^{-3}$, $\epsilon = 10^{-7}$); a cap of 20 epochs; early-stopping patience of 3; Glorot-uniform weight initialization with zero biases in both frameworks (PyTorch is re-initialized to match Keras); BatchNorm momentum of 0.9 in Keras corresponds to 0.1 in PyTorch (both express running = $0.9 \cdot \text{running} + 0.1 \cdot \text{batch}$), with $\epsilon = 10^{-5}$; each PyTorch model’s trainable parameter count is asserted to match its Keras counterpart exactly.

Environment for the reported run. Windows, Python 3.13, TensorFlow 2.21, PyTorch 2.14 (CPU build). Both frameworks were run on the **CPU** (native Windows TensorFlow has no GPU support, and the PyTorch build used was CPU-only), so all speed comparisons are CPU-to-CPU. Per-epoch timing is the median across epoch 2 onward, since epoch 1 includes setup overhead.

4 Datasets

Table 5: Datasets (all from Kaggle, download < 300 MB each).

Code	Source	Samples	Classes	Input
SV SVHN	sahityasetu/street-view-house-numbers-images	99,289	10	$32 \times 32 \times 3$ photo
GT GTSRB	harbhajansingh21/german-traffic-sign-dataset	51,839	43	$32 \times 32 \times 3$ photo
DB Diabetes	Playground Series (<code>train.csv</code>)	S5E12 300,000 700,000	of 2	24 features $\rightarrow d = 42$

4.1 SVHN

Cropped digit images taken from Google Street View, where the label refers only to the centre digit and any neighbouring digits are treated as noise. The two MATLAB files are read via `scipy.io.loadmat`, reshaped from $(32, 32, 3, N)$ to $(N, 32, 32, 3)$, and label 10 is remapped to digit 0. The classes are imbalanced: digit 1 accounts for 13,272 images versus 4,378 for digit 9 (a 3.0 ratio). Pixel values are rescaled via $x/255$ ($0 \rightarrow 0$, $128 \rightarrow 0.50$, $255 \rightarrow 1$); Keras consumes $(N, 32, 32, 3)$ while PyTorch consumes $(N, 3, 32, 32)$.

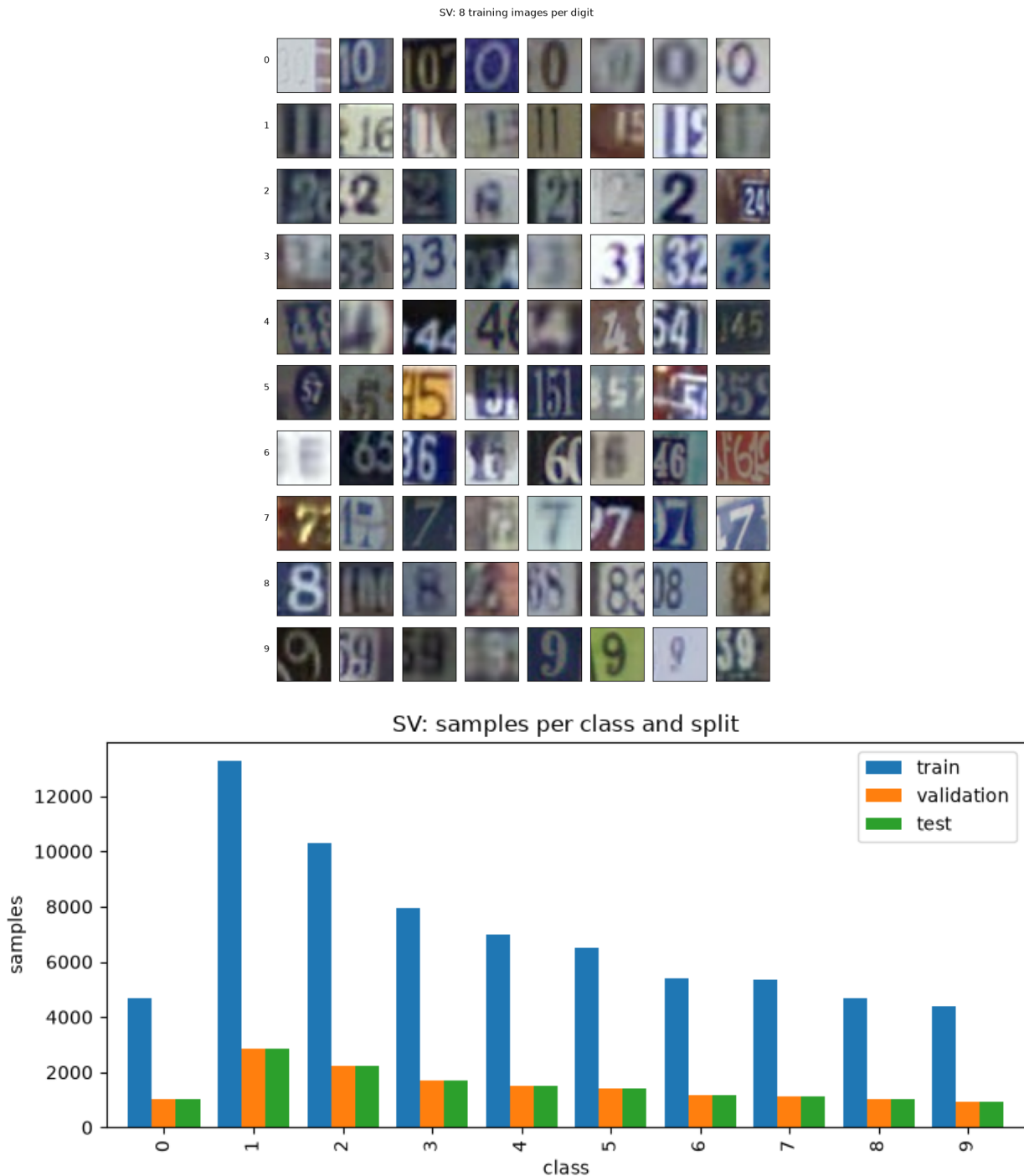


Figure 3: SV: 8 training images per digit (top) and images per class and split (bottom).

4.2 GTSRB

Traffic-sign photographs shot from a moving vehicle and resized to 32×32 . The three pickle files supplied are merged and then re-split. Class imbalance is pronounced (class 0, “Speed limit 20”, has just 189 images versus 2,100 for class 2, “Speed limit 50”, an 11.1 ratio), which is why macro F1 serves as the headline metric. A large share of the photos are quite dark, an effect that BatchNorm helps offset.



Figure 4: GT: one training image per class (top) and images per class and split (bottom).

4.3 Diabetes (Playground Series S5E12)

This dataset holds 700,000 synthetic patient records generated by Kaggle from a real-world source, from which a stratified sample of 300,000 is drawn. Every row is converted into $d = 42$ numbers: 15 numeric features standardized using the training statistics ($z = (x - \mu)/\sigma$, e.g. BMI $31.7 \rightarrow (31.7 - 25.9)/2.9 = 2.0$), 6 categorical features one-hot encoded, and 3 binary flags retained as-is. Since 62.3% of patients are diabetic, class weights $w_c = N/(2N_c)$ are applied: $w_0 = 1.327$, $w_1 = 0.802$. The most discriminating numeric features (age, physical activity) separate the two classes by only about 0.4 standard deviations, an early hint that this will be a difficult prediction task.

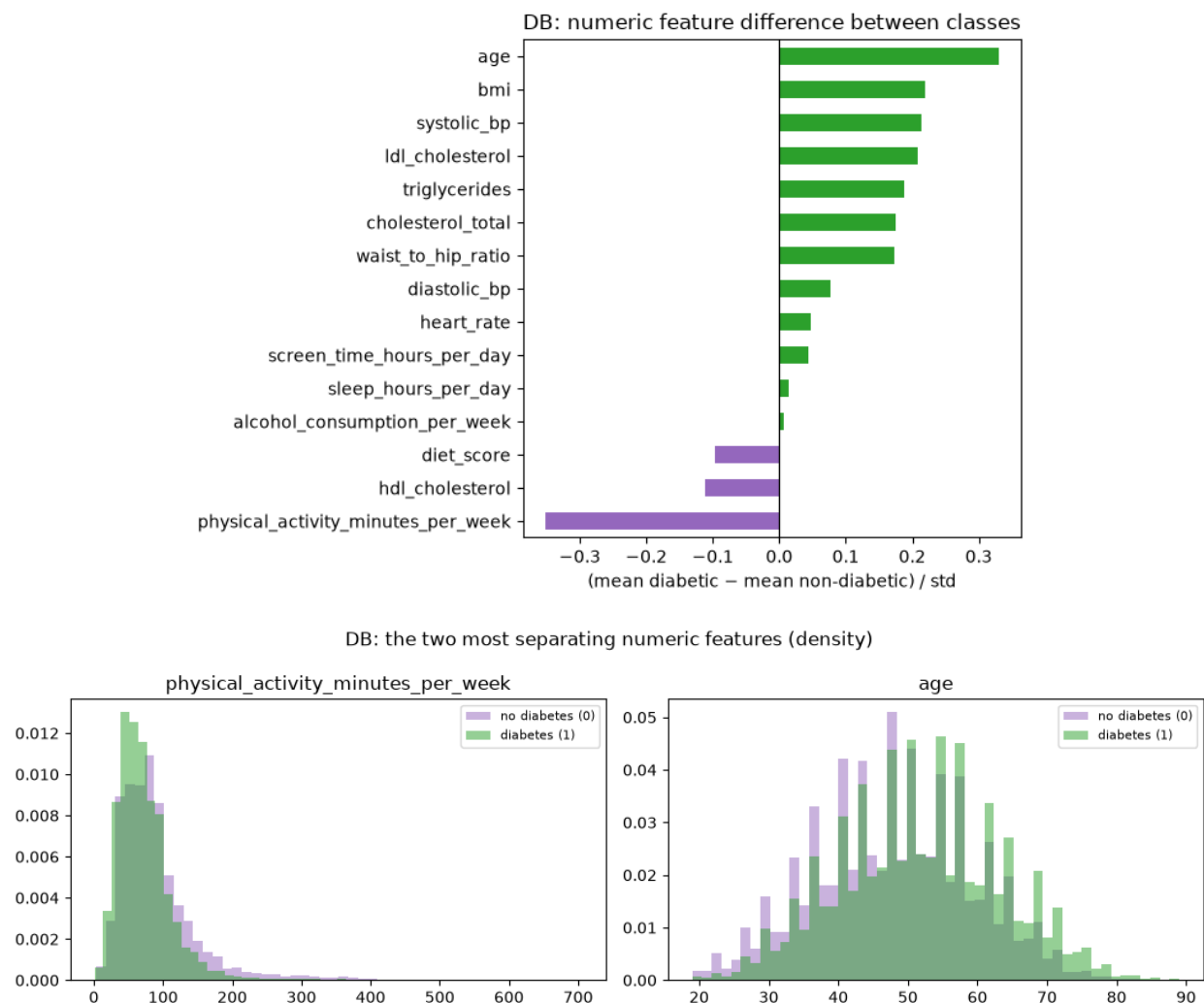


Figure 5: DB: standardized mean difference per numeric feature (top) and the two most separating features (bottom).

5 Models

Table 6: Image models (SVHN, GTSRB). 3×3 filters, padding “same”; each Conv is followed by BatchNorm and ReLU.

Step	CNN-3 output	CNN-5 output
input	$32 \times 32 \times 3$	$32 \times 32 \times 3$
block 1	Conv 32 $\rightarrow$ Pool $\rightarrow 16 \times 16 \times 32$	Conv 32 $\rightarrow$ Conv 32 $\rightarrow$ Pool $\rightarrow 16 \times 16 \times 32$
block 2	Conv 64 $\rightarrow$ Pool $\rightarrow 8 \times 8 \times 64$	Conv 64 $\rightarrow$ Conv 64 $\rightarrow$ Pool $\rightarrow 8 \times 8 \times 64$
block 3	Conv 128 $\rightarrow$ Pool $\rightarrow 4 \times 4 \times 128$	Conv 128 $\rightarrow$ Pool $\rightarrow 4 \times 4 \times 128$
head	GAP $\rightarrow$ Dropout 0.3 $\rightarrow$ Dense 256 $\rightarrow$ Dense 10/43	same
receptive field	22×22	28×28

Table 7: Tabular models (diabetes). Conv1D kernel 3, padding “same”, no pooling.

Step	CNN-3	CNN-5
Conv1D filters	$32 \rightarrow 64 \rightarrow 64$ ($d \times 64$)	$32 \rightarrow 32 \rightarrow 64 \rightarrow 64 \rightarrow 128$ ($d \times 128$)
head	Flatten $\rightarrow$ Dropout 0.3 $\rightarrow$ Dense 64 $\rightarrow$ Dense 1	same
features one position sees	7	11

Table 8: Trainable parameters (identical in TF and PT).

Model	SVHN	GTSRB	Diabetes
CNN-3	129,290	137,771	191,169
CNN-5	175,658	184,139	391,329
MLP baseline	—	—	6,977

5.1 TensorFlow, Keras and PyTorch

- **TensorFlow** (Google) is the lower-level library: it handles tensors, automatic differentiation, and CPU/GPU execution.
- **Keras** sits *inside* TensorFlow as its high-level API — a model is declared layer by layer and trained with `compile + fit`, while TensorFlow does the actual computing underneath.
- **PyTorch** (Meta) is a separate library that operates at roughly the same level as TensorFlow itself.

In other words, “TensorFlow” and “Keras” name one side of this comparison (models written in Keras, run by TensorFlow, run ID TF), and PyTorch names the other (run ID PT).

Table 9: How the same CNN is written and trained in the two frameworks.

	Keras / TensorFlow	PyTorch
model	layers stacked declaratively: <code>Conv2D(32, 3, padding="same")</code>	<code>nn.Module</code> subclass: <code>nn.Conv2d(3, 32, 3, padding=1)</code> , input channels specified manually
image axes	$(N, 32, 32, 3)$, channels last	$(N, 3, 32, 32)$, channels first
training	single <code>compile(...)</code> + <code>fit(...)</code> call	a manually written loop: <code>zero_grad</code> → forward → loss → <code>backward</code> → <code>step</code>
early stopping	handled by the built-in <code>EarlyStopping</code> callback	tracked with a counter inside the loop
train / test mode	toggled automatically by <code>fit</code> / <code>predict</code>	<code>model.train()</code> / <code>model.eval()</code> invoked explicitly
device	picks up a GPU automatically when present	requires <code>.to(device)</code> on the model and every batch
strength	concise, fast to write	every step is explicit and easy to modify
results here	comparable accuracy (gap under 0.011 with matching architecture and settings); in this CPU run Keras finished 9–16% faster per epoch on the image datasets and about 50% faster on the tabular one	

5.2 Implementation (TensorFlow/Keras)

Table 10: Keras building blocks used in the image models.

Step	Keras / TensorFlow
data	<code>tf.data.Dataset: shuffle → batch(256) → divide by 255 → prefetch;</code> images ($N, 32, 32, 3$)
conv block	<code>Conv2D(F, 3, padding="same") →</code> <code>BatchNormalization(momentum=0.9) → ReLU() → MaxPooling2D(2)</code>
head	<code>GlobalAveragePooling2D() → Dropout(0.3) → Dense(256,</code> <code>activation="relu") → Dense(n_classes)</code>
init	Glorot uniform weights, zero biases (Keras default)
loss / optimizer	<code>SparseCategoricalCrossentropy(from_logits=True)</code> (diabetes: <code>BinaryCrossentropy</code>), <code>Adam(1e-3)</code>
early stopping	<code>EarlyStopping(monitor="val_loss", patience=3,</code> <code>restore_best_weights=True)</code>

```
def build_tf(arch, input_shape, n_classes):
    inputs = keras.Input(shape=input_shape) # (32, 32, 3)
    x = inputs
    for filters, pool in arch: # e.g. [(32, True), (64, True), (128, True)]
        x = layers.Conv2D(filters, 3, padding="same")(x)
        x = layers.BatchNormalization(momentum=0.9, epsilon=1e-5)(x)
        x = layers.ReLU()(x)
        if pool:
            x = layers.MaxPooling2D(2)(x)
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dropout(0.3)(x)
    x = layers.Dense(256, activation="relu")(x)
    return keras.Model(inputs, layers.Dense(n_classes)(x)) # logits

model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=["accuracy"])
model.fit(train_ds, validation_data=val_ds, epochs=20,
          callbacks=[keras.callbacks.EarlyStopping(monitor="val_loss", patience=3,
                                                    restore_best_weights=True)])
```

A single call to `fit` executes the entire training loop from Section 2.6 (batching, forward pass, backward pass, Adam update, validation, early stopping); the PyTorch version spells these same steps out explicitly (see below).

5.3 Implementation (PyTorch)

Here is the equivalent model built in PyTorch. Its layers and settings match the Keras version exactly (identical parameter counts, confirmed in the notebooks), but the training loop itself is written by hand.

Table 11: PyTorch building blocks used in the image models.

Step	PyTorch
data	<code>TensorDataset + DataLoader(batch_size=256, shuffle=True)</code> ; images ($N, 3, 32, 32$), divided by 255 per batch
conv block	<code>Conv2d(C_in, F, 3, padding=1) → BatchNorm2d(F) → ReLU() → MaxPool2d(2)</code>
head	<code>AdaptiveAvgPool2d(1) → Flatten() → Dropout(0.3) → Linear(128, 256) → ReLU() → Linear(256, n_classes)</code>
init	<code>xavier_uniform_ weights</code> , zero biases
loss / optimizer	<code>CrossEntropyLoss()</code> (diabetes: <code>BCEWithLogitsLoss</code>), <code>Adam(lr=1e-3)</code>
early stopping	keep the best <code>state_dict</code> ; stop after 3 epochs without a lower validation loss

```

class CNN(nn.Module):
    def __init__(self, arch, in_channels, n_classes):
        super().__init__()
        blocks, c = [], in_channels
        for filters, pool in arch: # e.g. [(32, True), (64, True), (128, True)]
            blocks += [nn.Conv2d(c, filters, 3, padding=1), nn.BatchNorm2d(filters), nn.
                ReLU()]
            if pool:
                blocks.append(nn.MaxPool2d(2))
            c = filters
        self.features = nn.Sequential(*blocks)
        self.head = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(), nn.Dropout(0.3),
            nn.Linear(c, 256), nn.ReLU(), nn.Linear(256, n_classes))

    def forward(self, x):
        return self.head(self.features(x))

for x, y in train_loader: # one epoch
    optimizer.zero_grad() # clear gradients
    loss = criterion(model(x), y) # forward + cross-entropy
    loss.backward() # backpropagation
    optimizer.step() # Adam update

```

6 Results per Dataset

How to read the figures

- **Training curves** plot loss and accuracy per epoch for both the training and validation sets. Both curves falling together means the model is still learning; training loss falling while validation loss climbs is **overfitting**, and early stopping stops the run at the epoch just before that starts.
- **Confusion matrix** (row-normalized). Rows are the true class, columns the predicted one, and each row sums to 1, so the diagonal entries are each class's recall. In Figure 7, for instance, row 7 / column 1 = 0.032 means about 3% of the real 7s got classified as 1.
- **Feature maps** are grayscale images of one filter's output — bright pixels show where that filter's pattern was detected.
- **ROC curve** (diabetes) plots true-positive rate against false-positive rate across all thresholds. The diagonal is random guessing (AUC 0.5); the top-left corner is a perfect classifier (AUC 1).

6.1 SVHN

Table 12: SV test results.

Run ID	Accuracy	Macro F1	Epochs	s/epoch	Train time (s)	ms / 1k pred.
SV-TF-CNN-3	0.9012	0.8959	14	19.8	279	0.07
SV-TF-CNN-5	0.9398	0.9363	20	42.9	865	0.14
SV-PT-CNN-3	0.9091	0.9069	20	28.1	557	0.15
SV-PT-CNN-5	0.9319	0.9282	12	61.5	750	0.27

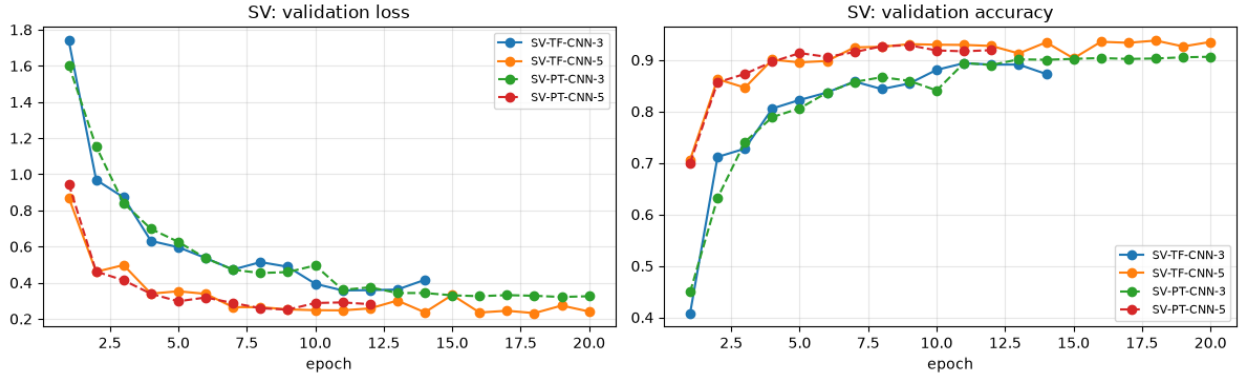


Figure 6: SV: validation loss and accuracy of the four runs (solid = TF, dashed = PT).

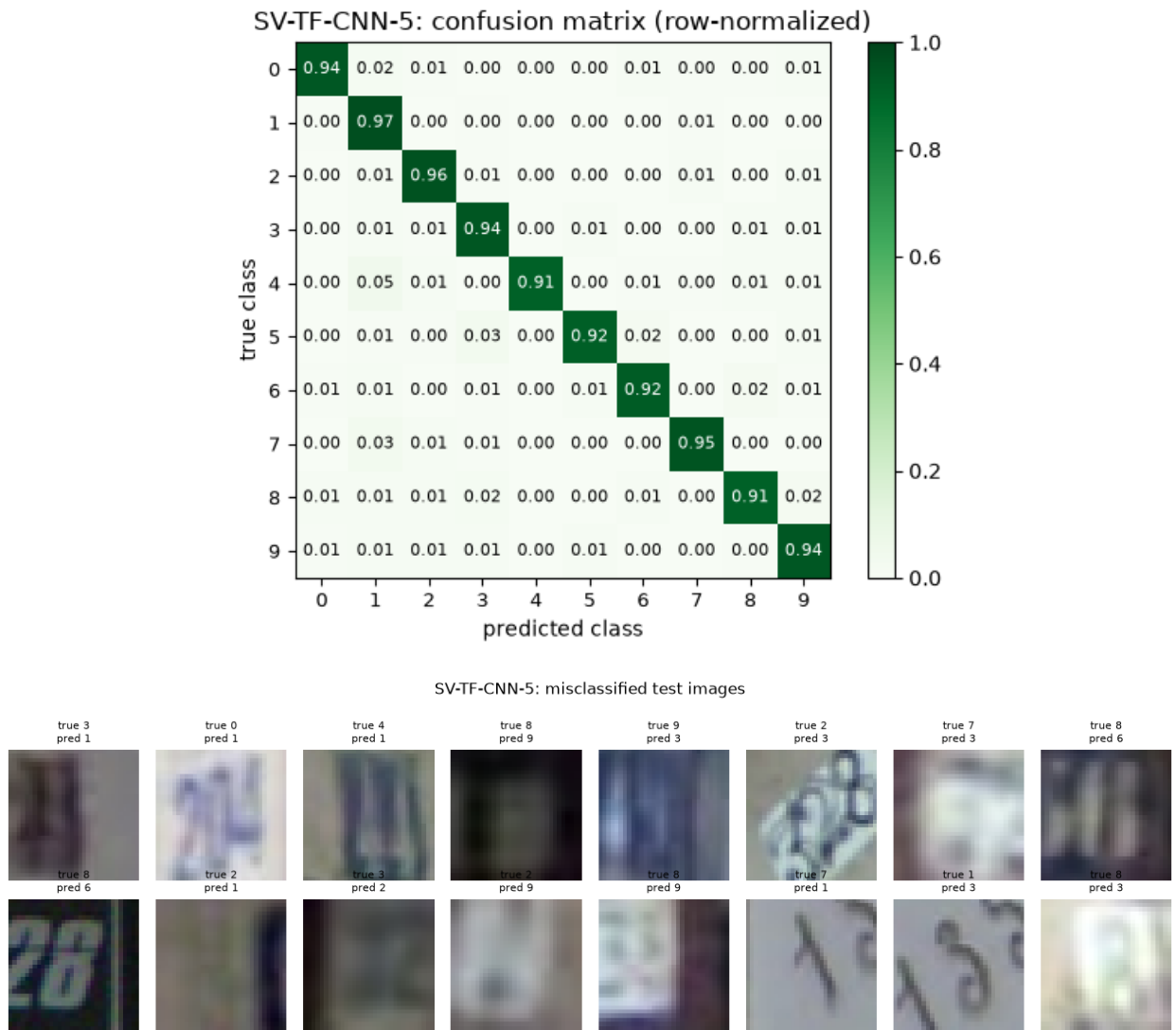


Figure 7: SV-TF-CNN-5: confusion matrix (top) and misclassified test images (bottom).

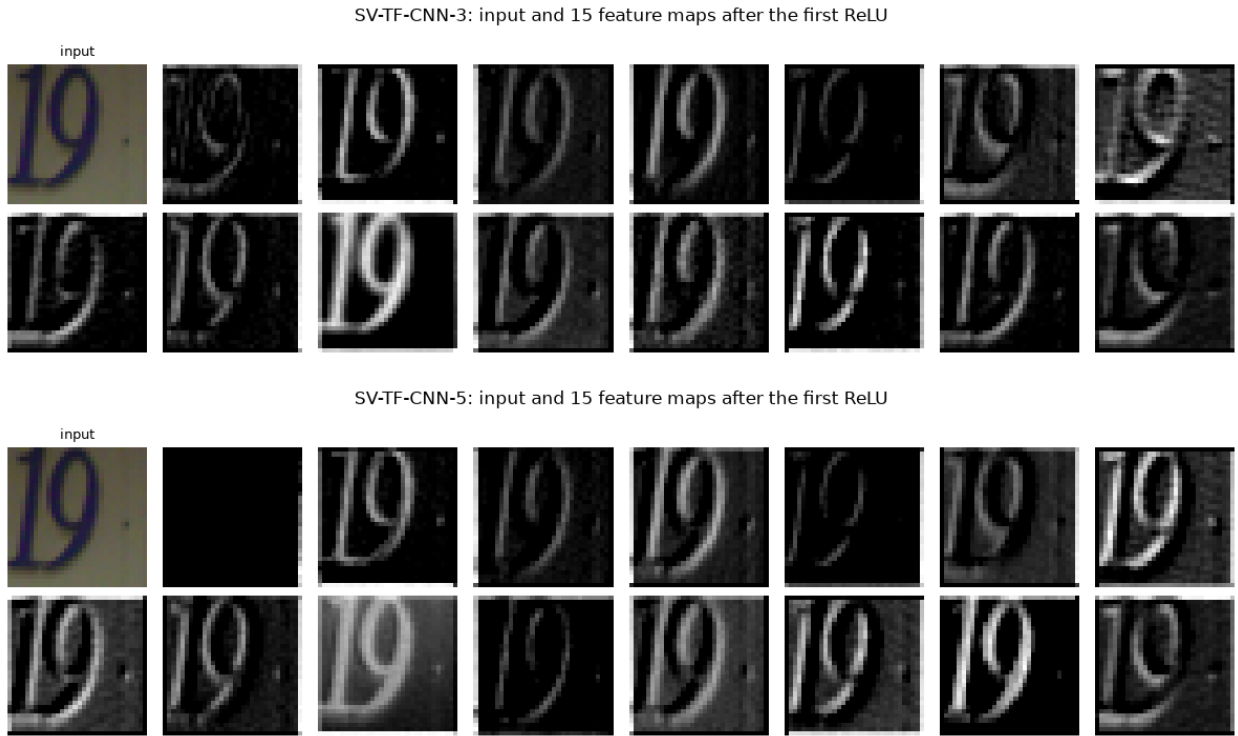


Figure 8: First-layer feature maps after ReLU for one test image: SV-TF-CNN-3 (top), SV-TF-CNN-5 (bottom).

- **Depth.** Moving to CNN-5 raises macro F1 by +0.040 (TF) and +0.021 (PT), and accuracy by +0.039 (TF) and +0.023 (PT). Under PyTorch it also reaches its best validation loss sooner (epoch 9 versus 19), though every epoch costs roughly twice as much time either way.
- **Framework.** At CNN-3, TF trails PT by about 0.008 accuracy; at CNN-5 that flips and TF leads PT by about the same margin.
- **Errors.** On the best model (SV-TF-CNN-5), the most common mistake is a 4 read as a 1 (4.6% of the 4s), then 5 read as 3 (3.4%) and 7 read as 1 (3.2%) — all digits that share a vertical stroke. Looking through the misclassified-image grid, the usual suspects show up: blurry crops, low contrast, and digits crowded by their neighbours.
- The first-layer feature maps mostly respond to the digit’s strokes and edges, though a few pick up on background colour instead.

6.2 GTSRB

Table 13: GT test results.

Run ID	Accuracy	Macro F1	Epochs	s/epoch	Train time (s)	ms / 1k pred.
GT-TF-CNN-3	0.9801	0.9745	20	10.2	207	0.07
GT-TF-CNN-5	0.9924	0.9893	12	21.9	268	0.15
GT-PT-CNN-3	0.9685	0.9621	14	14.2	199	0.15
GT-PT-CNN-5	0.9973	0.9979	19	30.0	572	0.26

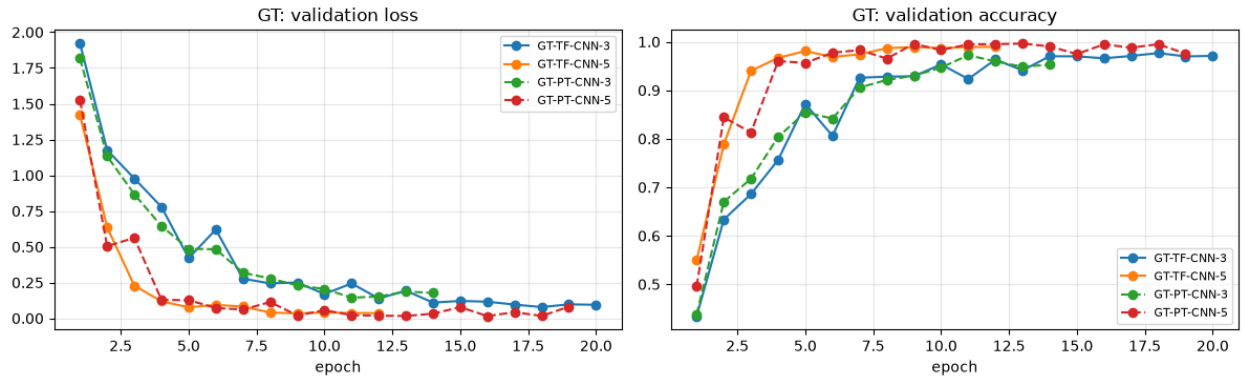


Figure 9: GT: validation loss and accuracy of the four runs (solid = TF, dashed = PT).

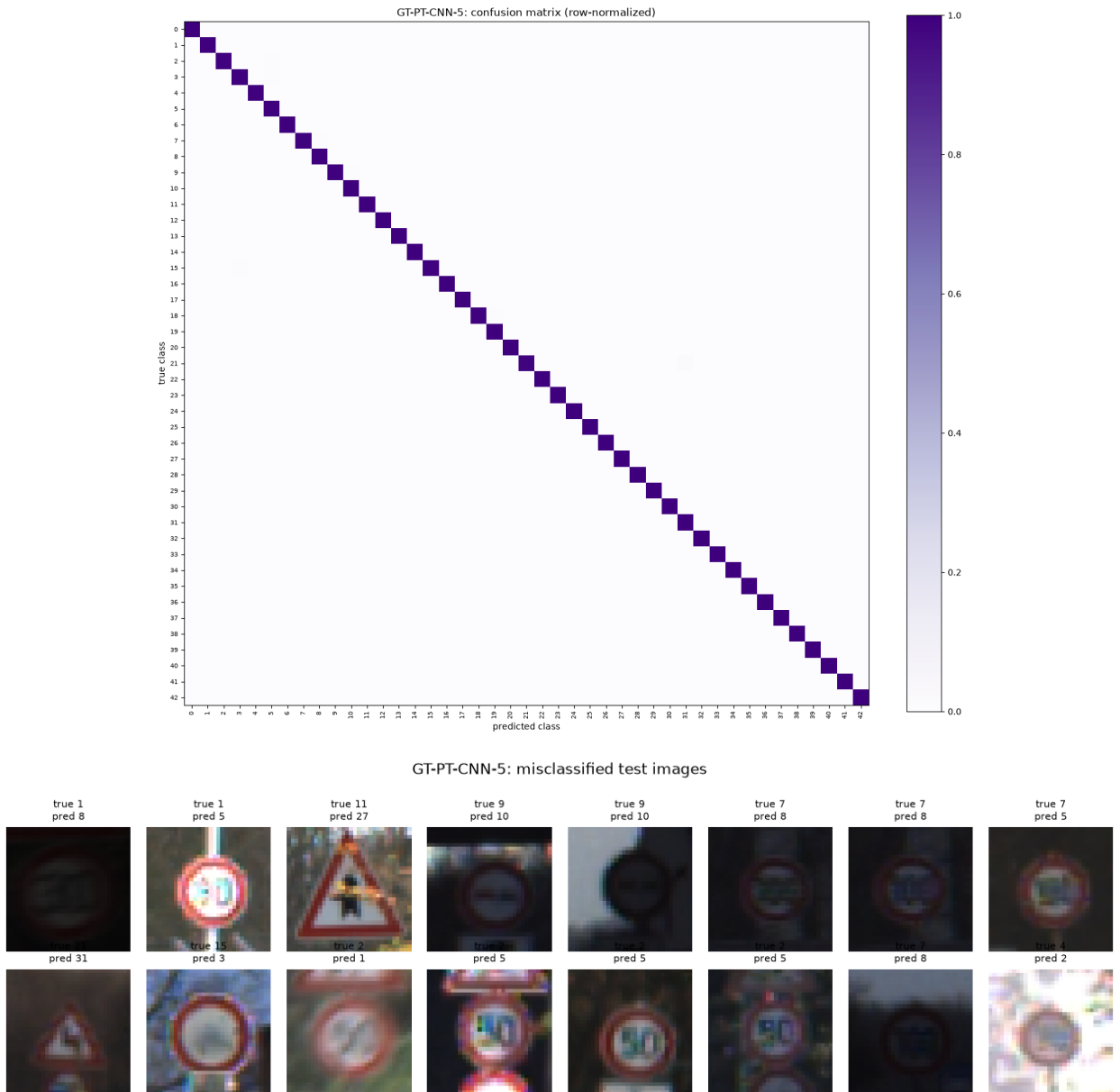


Figure 10: GT-PT-CNN-5: confusion matrix over 43 classes (top) and misclassified test signs (bottom).

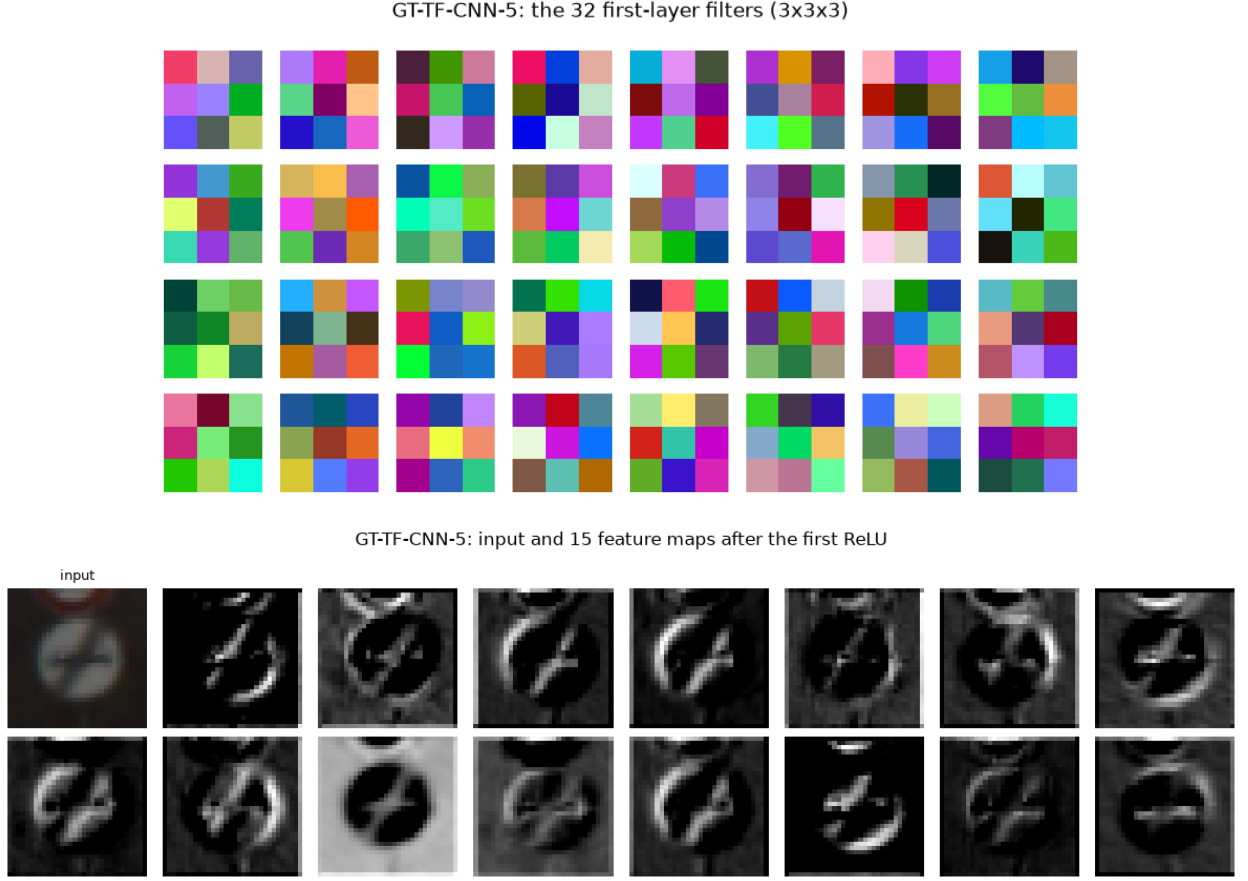


Figure 11: GT-TF-CNN-5: the 32 first-layer filters (top) and feature maps for one test sign (bottom).

- **Depth.** CNN-5 raises macro F1 by +0.015 (TF) and +0.036 (PT). Macro F1 weighs all 43 classes equally, so this gain is not just coming from the common speed-limit signs — the rare sign types improve too.
- **Framework.** The winner flips with depth. At CNN-3, TF leads PT by 0.012 macro F1 (TF used the full 20 epochs to get there; PT stopped at epoch 14). At CNN-5, PT leads TF by 0.009 macro F1, and GT-PT-CNN-5 ends up the strongest model in the whole study.
- The confusion matrix is almost entirely diagonal. What the misclassified-image grid shows is mostly dark or blurred photos — the remaining hard cases.

6.3 Diabetes

Table 14: DB test results (threshold 0.5).

Run ID	Accuracy	Macro F1	F1 (pos.)	ROC-AUC	Params	Epochs	s/epoch
DB-TF-CNN-3	0.6452	0.6304	0.7045	0.6963	191,169	7	9.7
DB-TF-CNN-5	0.6358	0.6275	0.6831	0.6945	391,329	5	18.6
DB-PT-CNN-3	0.6400	0.6288	0.6934	0.6951	191,169	8	21.1
DB-PT-CNN-5	0.6454	0.6303	0.7051	0.6955	391,329	6	38.4
DB-TF-MLP (baseline)	0.6276	0.6221	0.6677	0.6946	6,977	5	1.2

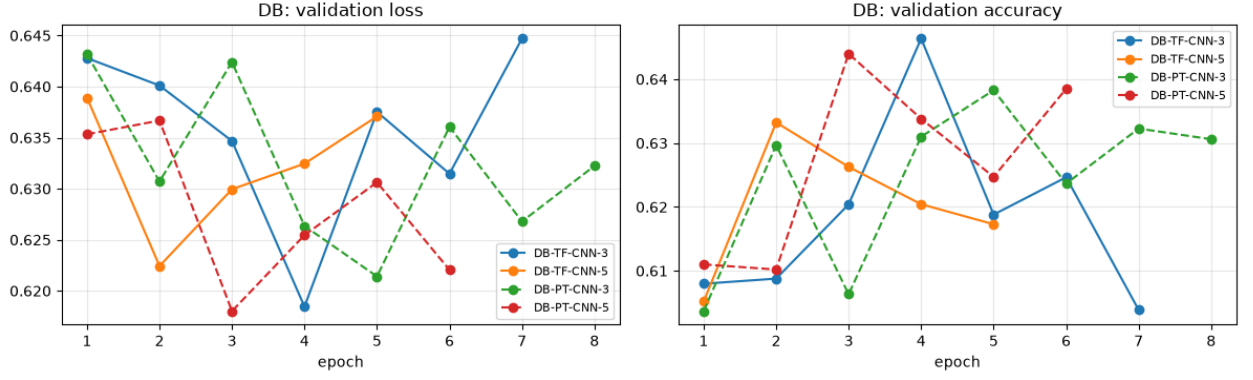


Figure 12: DB: validation loss and accuracy of the four runs (solid = TF, dashed = PT).

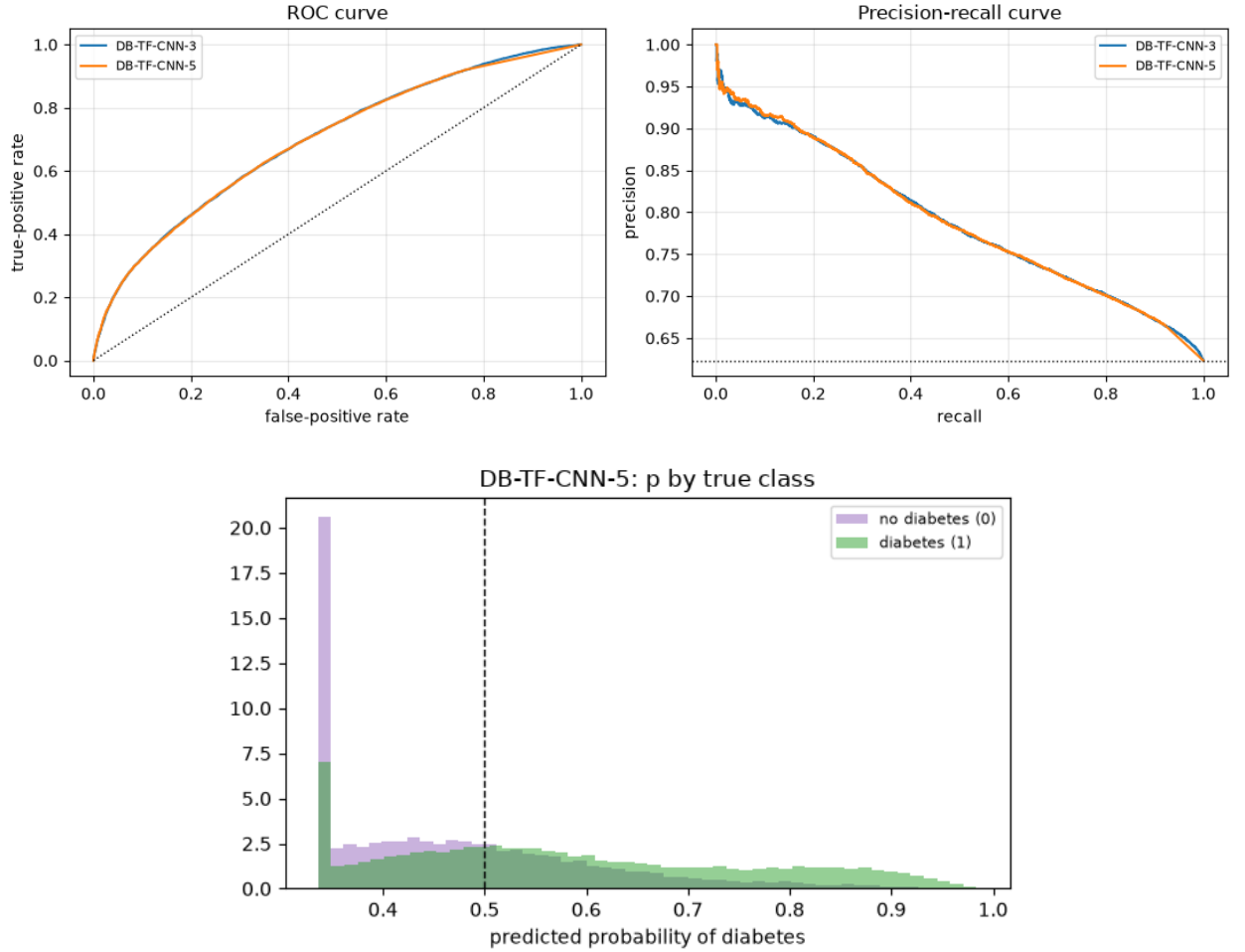


Figure 13: DB-TF: ROC and precision-recall curves (top); predicted probability by true class for DB-TF-CNN-5 (bottom).

- **Depth.** There is no consistent benefit here. Going from CNN-3 to CNN-5, ROC-AUC moves by -0.0018 (TF) and $+0.0004$ (PT), despite the parameter count doubling and per-epoch time growing by $1.8\text{--}1.9\times$.
- **Framework.** TF and PT stay within 0.0012 ROC-AUC of one another. The accuracy differences that do show up (± 0.01) just reflect where individual predictions happen to fall around the 0.5 threshold.
- **Baseline.** An MLP with $56\times$ fewer parameters still reaches ROC-AUC 0.6946, about the

same as the CNNs. The convolution’s core assumption — that neighbouring features carry information — does not seem to hold here.

- The validation curves oscillate within a narrow band (loss 0.62–0.645). With such a weak signal, every model settles into a noisy optimum after only 2–5 epochs, and early stopping cuts training short soon after that.
- The predicted-probability histogram shows the two classes overlapping heavily. Moving the decision threshold away from 0.5 would just trade precision for recall, not actually separate the classes.

7 Cross-Dataset Comparison

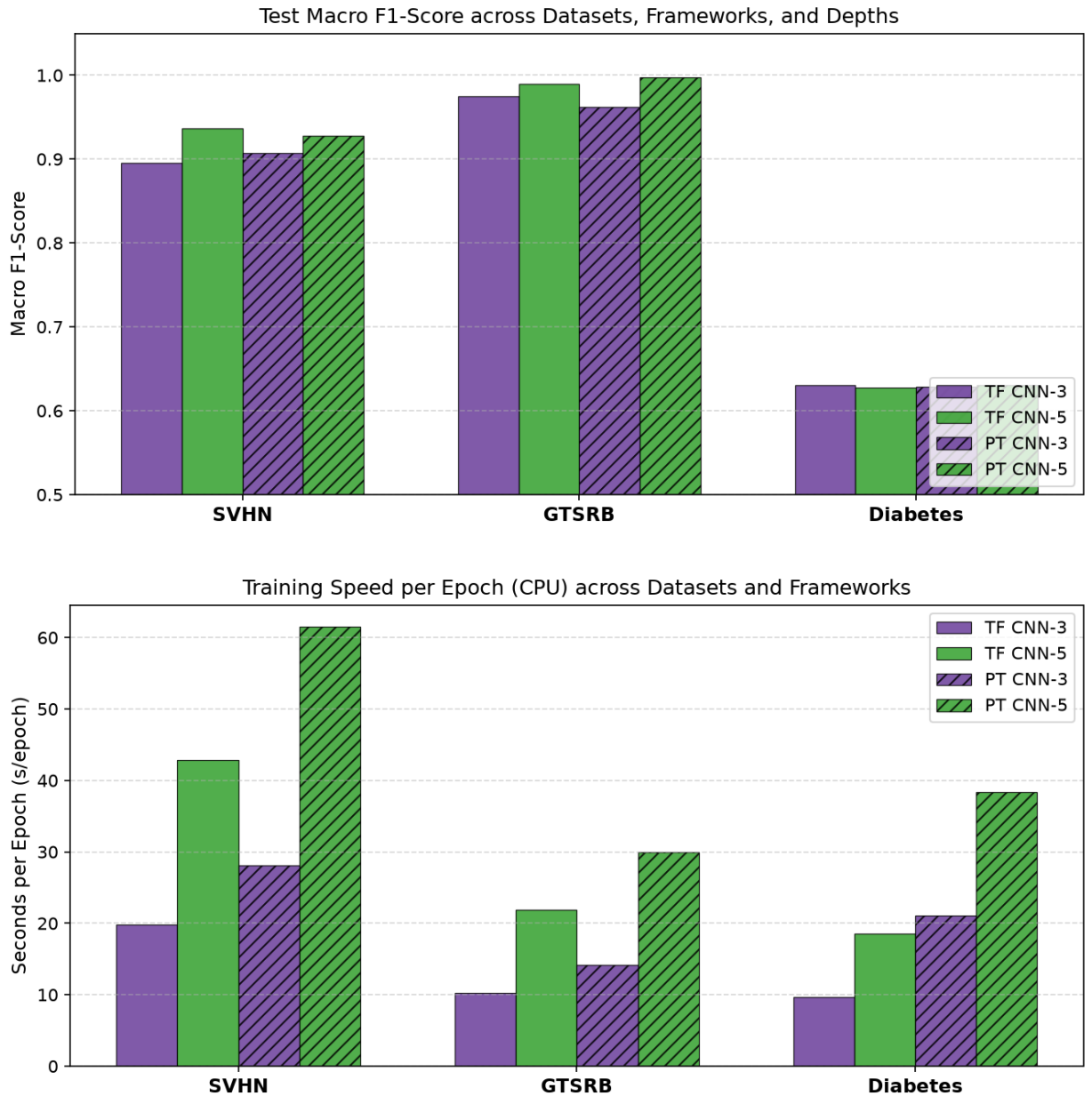


Figure 14: All 12 runs: test macro F1 (top) and training time per epoch (bottom). Purple = CNN-3, green = CNN-5; the hatched bars are PyTorch, the plain bars are TensorFlow/Keras.

Table 15: Effect of depth (CNN-5 minus CNN-3) and of framework (PT minus TF).

Comparison		SVHN	GTSRB	Diabetes
CNN-5 – CNN-3, macro F1	TF	+0.0404	+0.0148	−0.0029
	PT	+0.0213	+0.0358	+0.0015
CNN-5 / CNN-3, time per epoch	TF	2.17×	2.15×	1.92×
	PT	2.19×	2.11×	1.82×
PT – TF, macro F1	CNN-3	+0.0110	−0.0124	−0.0016
	CNN-5	−0.0081	+0.0086	+0.0028
PT / TF, time per epoch	CNN-3	1.42×	1.39×	2.18×
	CNN-5	1.43×	1.37×	2.06×

- Depth pays off wherever the input has spatial structure — +0.01 to +0.04 macro F1 on both image datasets — but changes nothing on the tabular data, where the ceiling is set by the features themselves and Conv1D has no real spatial structure to exploit.
- Both frameworks reach comparable quality: every PT–TF gap stays under 0.013 macro F1, about the same size as the noise you’d expect from run to run with a single seed — though which framework comes out ahead is not consistent, flipping between CNN-3 and CNN-5 on both image datasets. Where they clearly differ is speed. On this CPU run, Keras was quicker per epoch across the board, and the gap was largest on the tabular model, where PyTorch’s per-batch Python loop adds the most overhead (roughly 2× slower there versus 1.4× on the image datasets). Keras also predicted faster at inference time — roughly 1.7–2.1× per 1,000 images on the image datasets, 1.3–1.5× on the tabular data.
- CNN-5 costs roughly twice the time per epoch, and because it does not consistently converge in fewer epochs on this run, its total training time ends up anywhere from 1.3× to 3.1× that of CNN-3 on the image datasets, depending on the run.

7.1 Keras vs. PyTorch outputs

The same model trained under both frameworks, placed side by side (test set).

Dataset	Model	Keras / TensorFlow				PyTorch			
		Acc.	Macro F1	Epochs	s/epoch	Acc.	Macro F1	Epochs	s/epoch
SVHN	CNN-3	0.9012	0.8959	14	19.8	0.9091	0.9069	20	28.1
SVHN	CNN-5	0.9398	0.9363	20	42.9	0.9319	0.9282	12	61.5
GTSRB	CNN-3	0.9801	0.9745	20	10.2	0.9685	0.9621	14	14.2
GTSRB	CNN-5	0.9924	0.9893	12	21.9	0.9973	0.9979	19	30.0
Diabetes	CNN-3	0.6452	0.6304	7	9.7	0.6400	0.6288	8	21.1
Diabetes	CNN-5	0.6358	0.6275	5	18.6	0.6454	0.6303	6	38.4

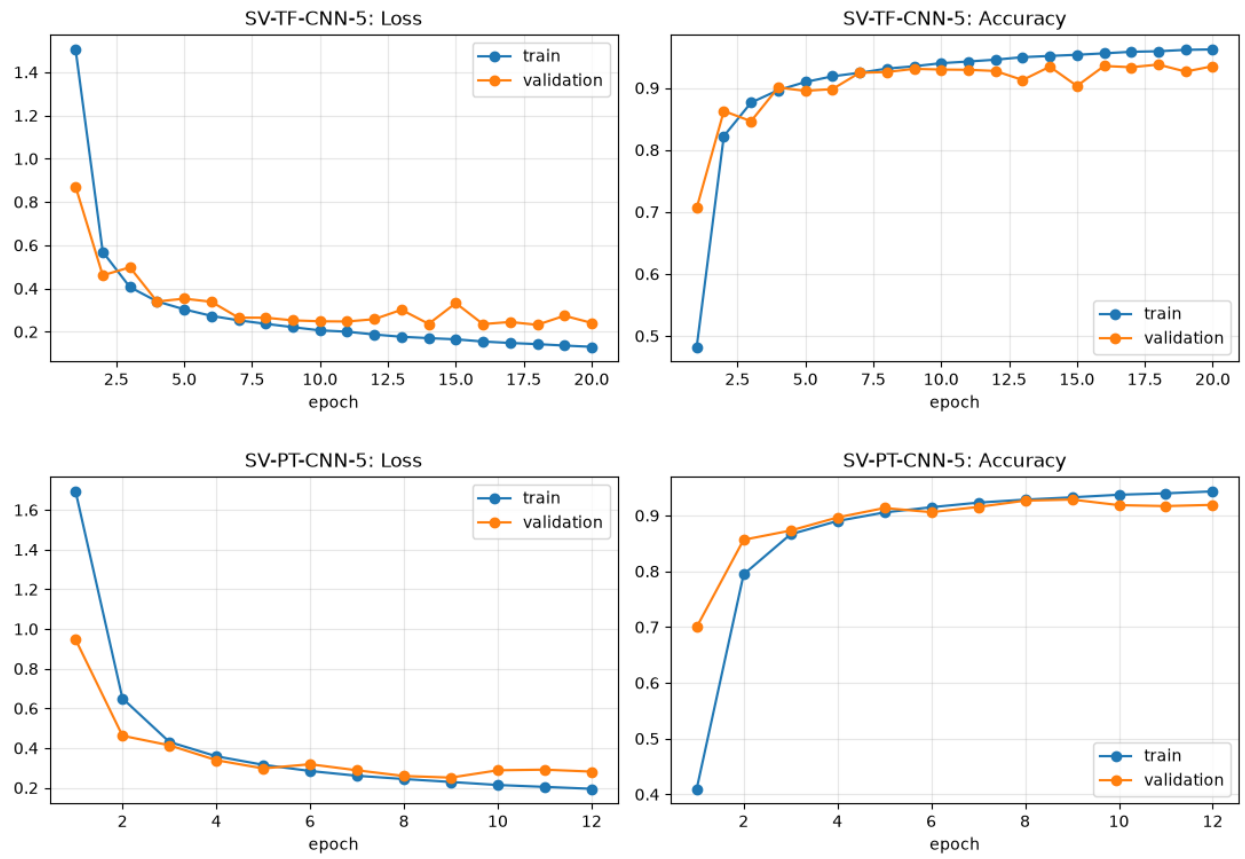


Figure 15: Training curves of CNN-5 on SVHN: SV-TF-CNN-5 (top) and SV-PT-CNN-5 (bottom).

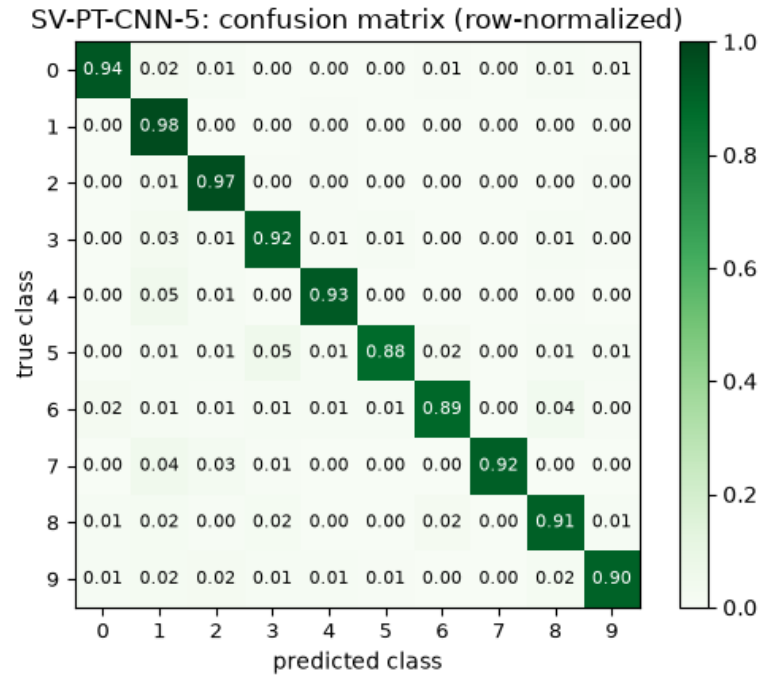
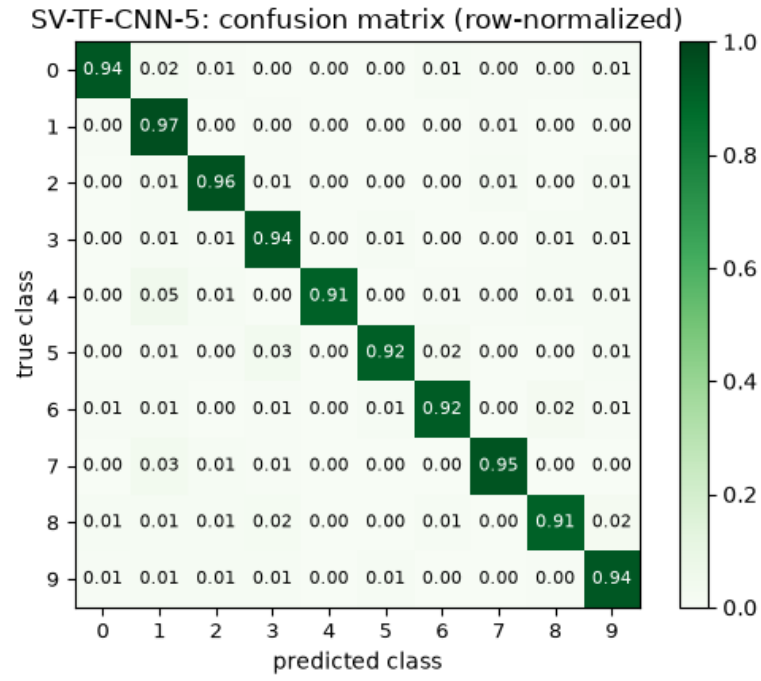


Figure 16: Confusion matrices of CNN-5 on SVHN: SV-TF-CNN-5 (top) and SV-PT-CNN-5 (bottom).

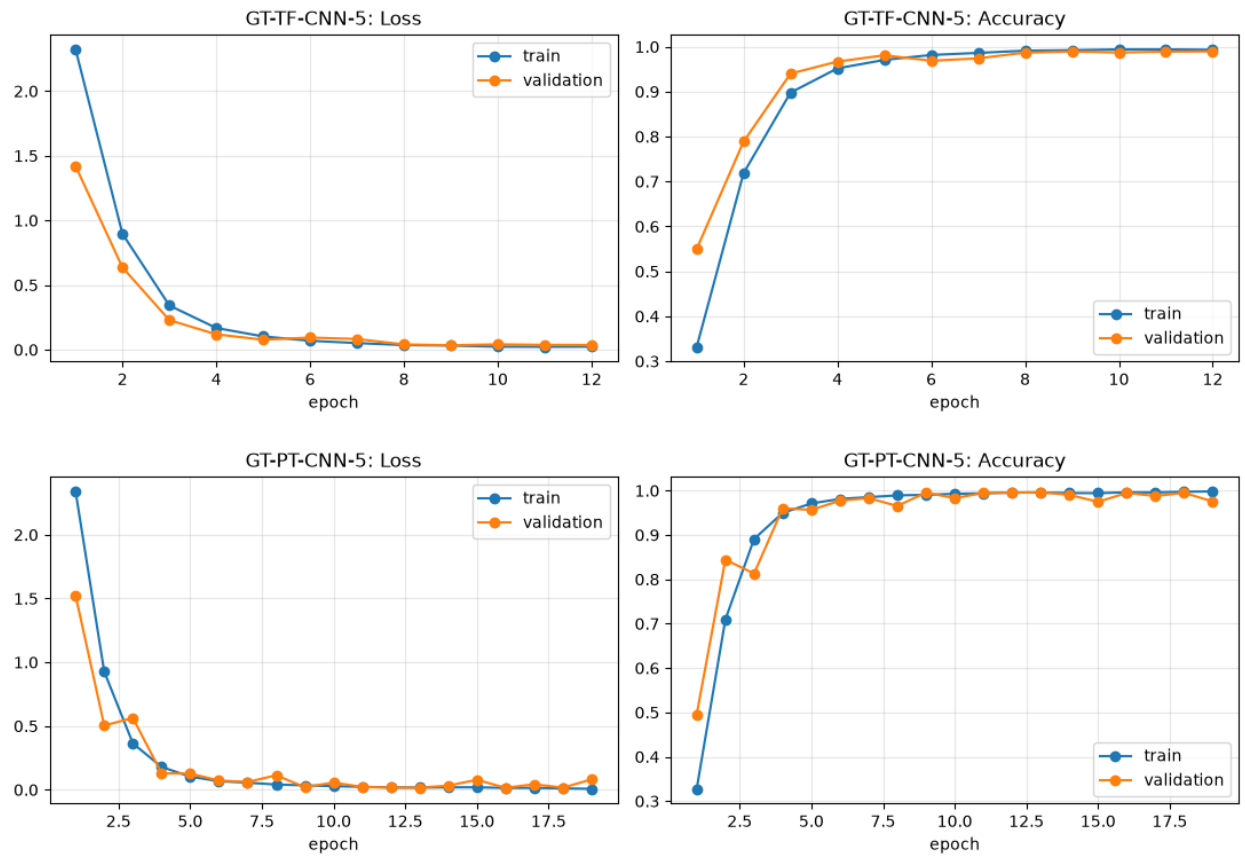


Figure 17: Training curves of CNN-5 on GTSRB: GT-TF-CNN-5 (top) and GT-PT-CNN-5 (bottom).

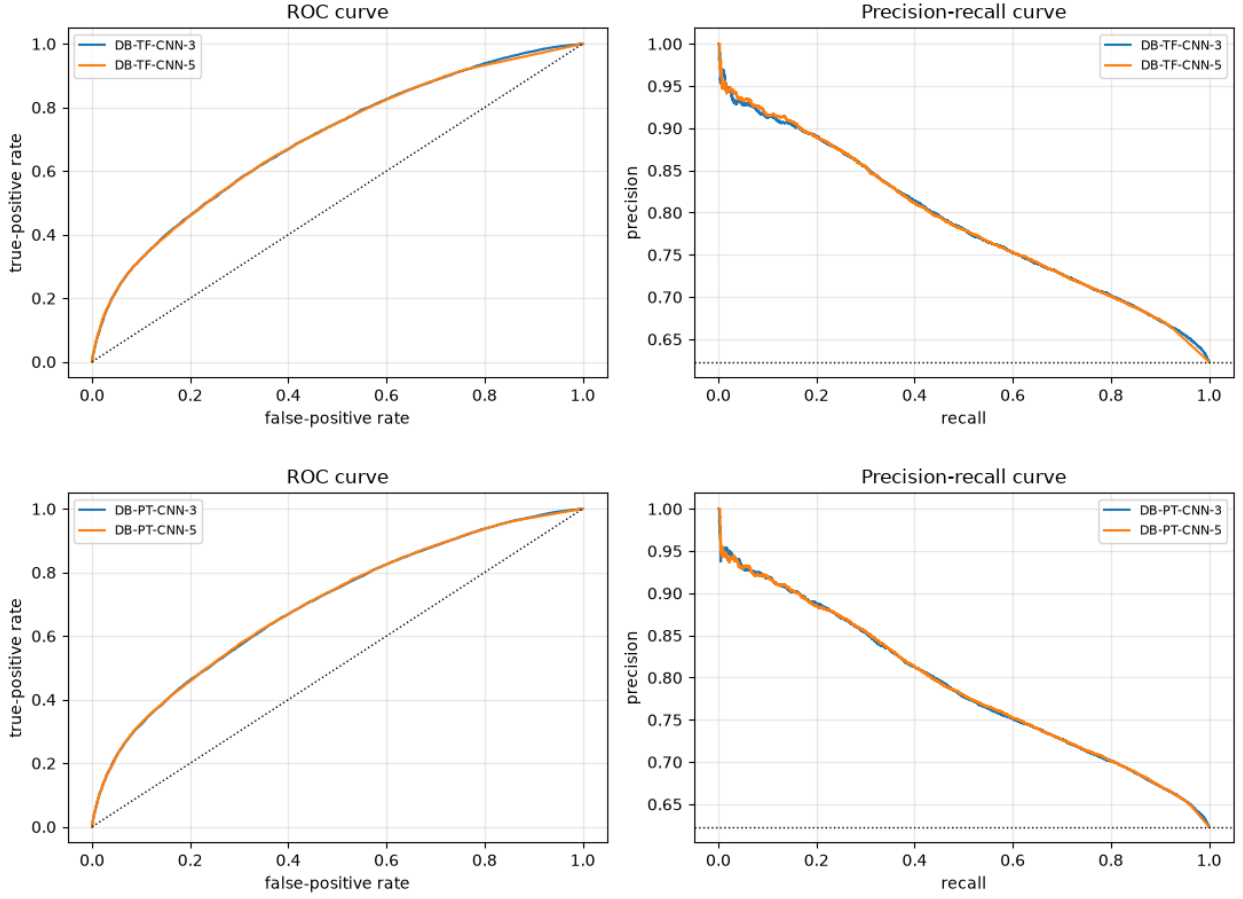


Figure 18: ROC and precision-recall curves on diabetes: Keras models (top) and PyTorch models (bottom).

7.2 Why depth helps on images: the receptive field

Every 3×3 convolution extends a unit's view by one pixel on each side, and every pooling operation doubles the stride between neighbouring units. Consequently the **receptive field** (the input region visible to one unit) grows as $RF \leftarrow RF + (K - 1) \cdot j$, with the jump j doubling each time pooling occurs:

CNN-3	conv1	+pool	conv2	+pool	conv3	+pool		
receptive field	3	4	8	10	18	22		
CNN-5	conv1	conv2	+pool	conv3	conv4	+pool	conv5	+pool
receptive field	3	5	6	10	14	16	24	28

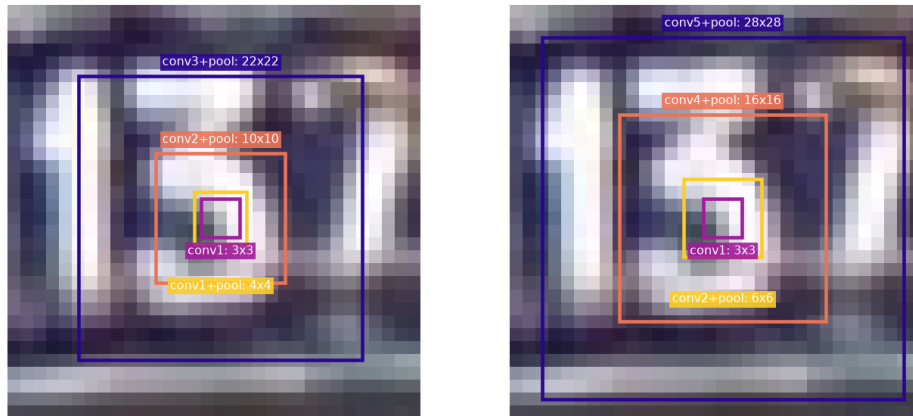
CNN-3: one final unit sees 22×22 of 32×32 pixelsCNN-5: one final unit sees 28×28 of 32×32 pixels

Figure 19: Receptive field after each block, drawn on an SVHN image. A final CNN-3 unit sees 22×22 pixels; a CNN-5 unit sees 28×28 , almost the whole digit and its neighbours.

Because of this, a CNN-5 unit can connect the top and bottom of a digit (for example, the open or closed left side distinguishing a 3 from a 5 or an 8), and it also benefits from two additional non-linear layers for combining patterns. This lines up with the observed results: $+0.02$ to $+0.04$ macro F1 on SVHN, where digits occupy most of the frame and have visual look-alikes, and $+0.015$ to $+0.036$ on GTSRB.

7.3 Why a CNN does not help on the diabetes table

	Image (SVHN, GTSRB)	Table (diabetes)
neighbours	adjacent pixels typically belong to the same stroke or edge	adjacent columns are unrelated to one another (e.g. <code>screen_time</code> sitting next to <code>bmi</code>); the ordering was simply our choice
same pattern, other place	an edge looks the same regardless of location, so one filter can be reused across the whole image	here position <i>is</i> the feature; a pattern found at “age” means something entirely different at “cholesterol”
signal in the data	strong: models reach 0.90–1.00	weak: the top feature separates the classes by only ≈ 0.4 standard deviations
result	CNN-5 > CNN-3	CNN-3 $\approx$ CNN-5 $\approx$ MLP (ROC-AUC 0.694–0.696)

The two premises that make convolution effective for images — nearby pixels matter, and the same filter is reusable everywhere — simply do not hold for tabular data. Adding depth or more filters therefore just adds parameters without adding capability; the true ceiling is set by how much information the 24 features actually contain, and every model arrives at that ceiling within 2–5 epochs.

8 Conclusion and Limitations

1. Going from CNN-3 to CNN-5 improves SVHN and GTSRB by 1.5 to 4 macro-F1 points. The extra two layers widen the receptive field from 22×22 to 28×28 , at roughly double the cost per epoch.
2. On the diabetes table, neither extra depth nor convolution itself helps — all four CNNs and even a 7k-parameter MLP land around ROC-AUC 0.695. A CNN is not a natural fit for this kind of tabular data.
3. The same architecture gives a broadly similar result in TensorFlow/Keras and PyTorch, with macro-F1 gaps under 0.013 on every dataset, though which framework comes out ahead is not consistent. That is compatible with the two frameworks differing mainly in how much they abstract away rather than in the underlying learning process, though a single seed per run cannot fully separate a real framework effect from ordinary training noise.
4. Keras needed less code to write and ran faster in this CPU setup; PyTorch trades that away for a training loop where every step is visible and easy to modify.

Limitations. Each run used a single seed (so no variance estimate is available); GPU/CPU kernels are not fully deterministic; no data augmentation was applied; the diabetes dataset is synthetic; Conv1D’s results depend on an arbitrary column ordering; all timings come from one CPU-only Windows machine; and three of the twelve image-model runs (two of them CNN-3) hit the 20-epoch cap without a clear validation-loss plateau.

A Appendix

A.1 Additional figures

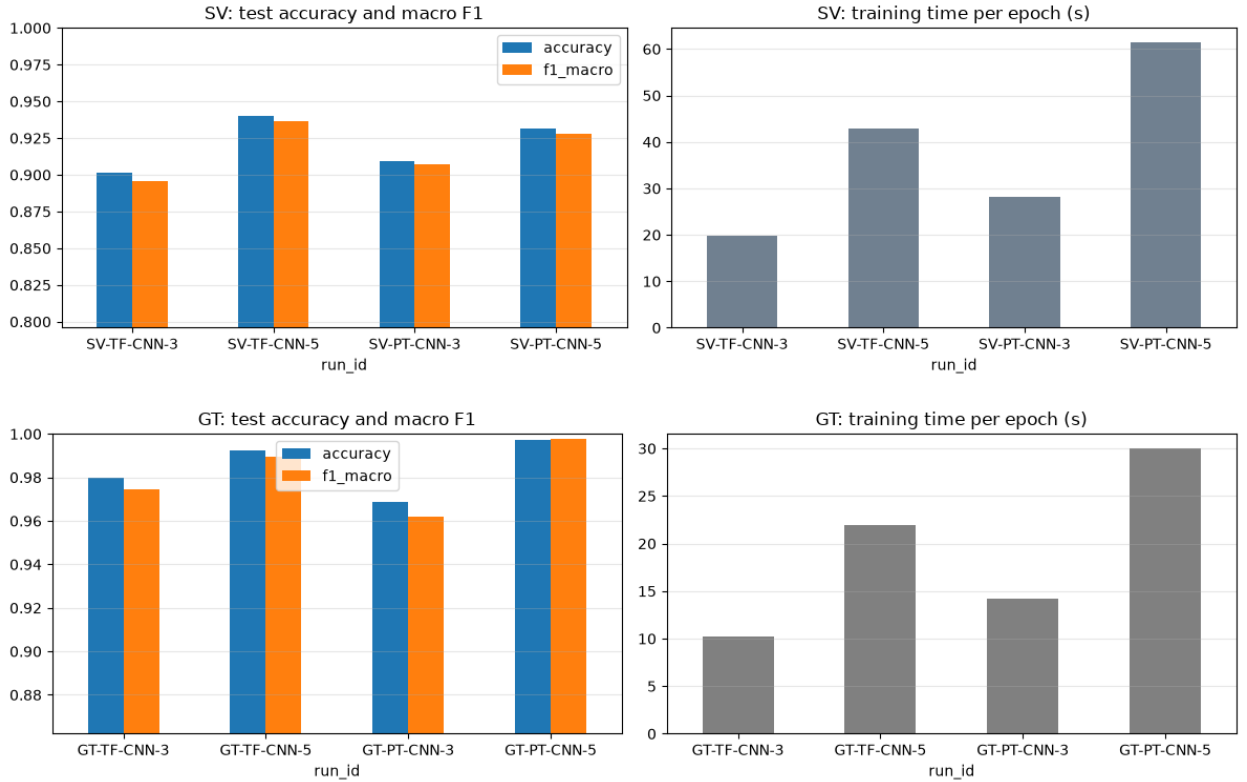


Figure 20: Per-dataset metric and time bars: SV (top), GT (bottom).

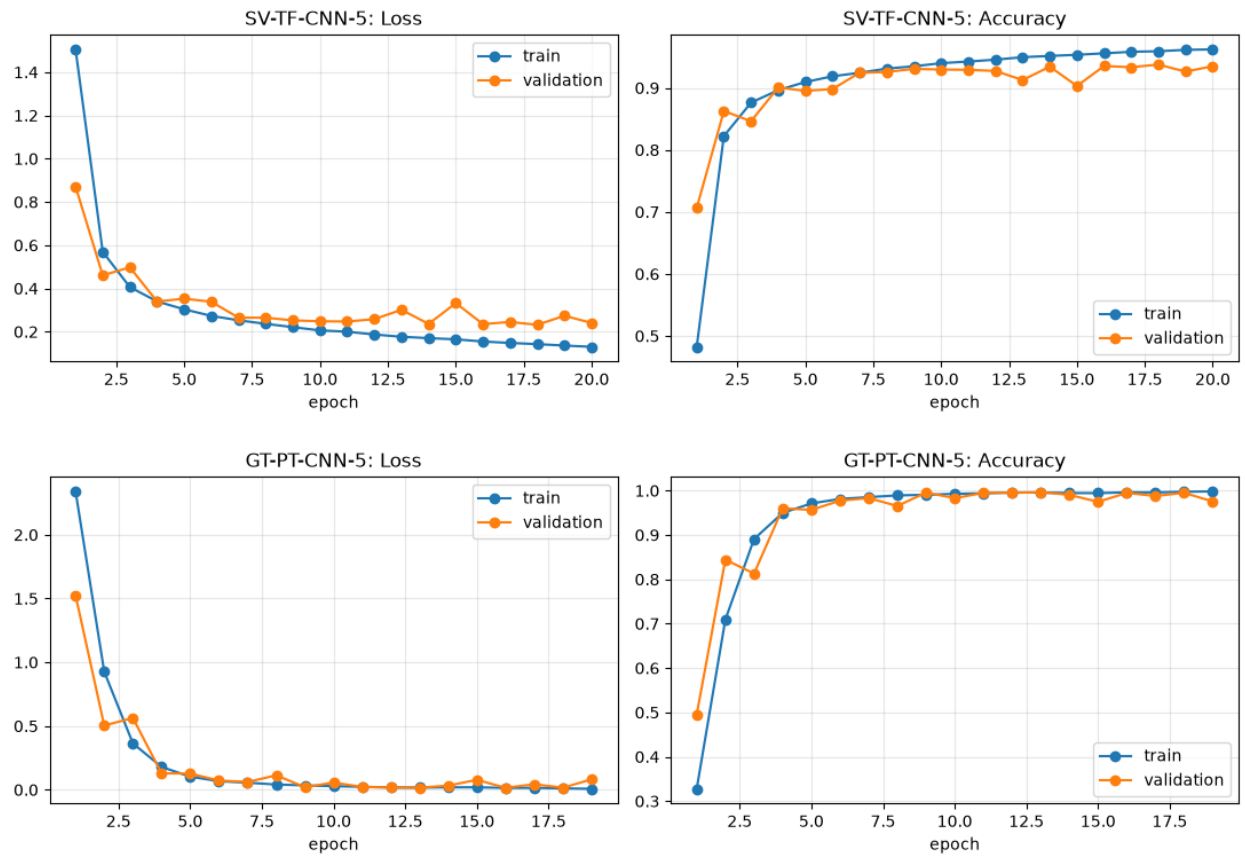
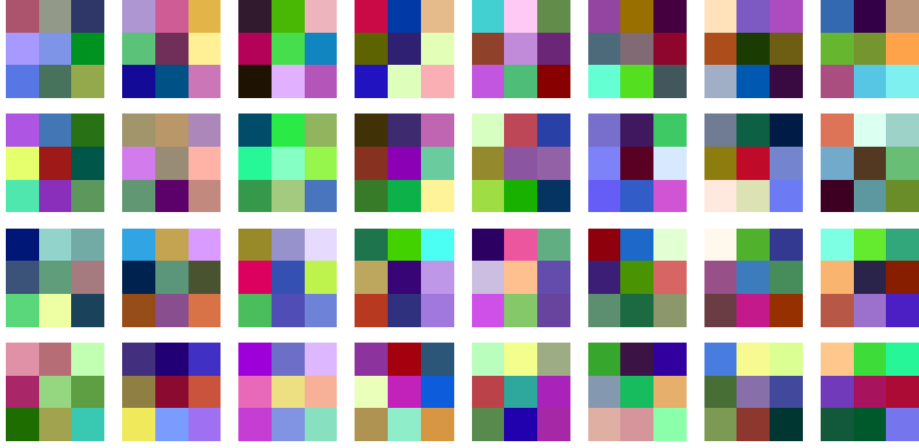


Figure 21: Training curves of the best image models: SV-TF-CNN-5 (top), GT-PT-CNN-5 (bottom).

SV-TF-CNN-5: the 32 first-layer filters (3x3x3)



SV-PT-CNN-5: the 32 first-layer filters (3x3x3)

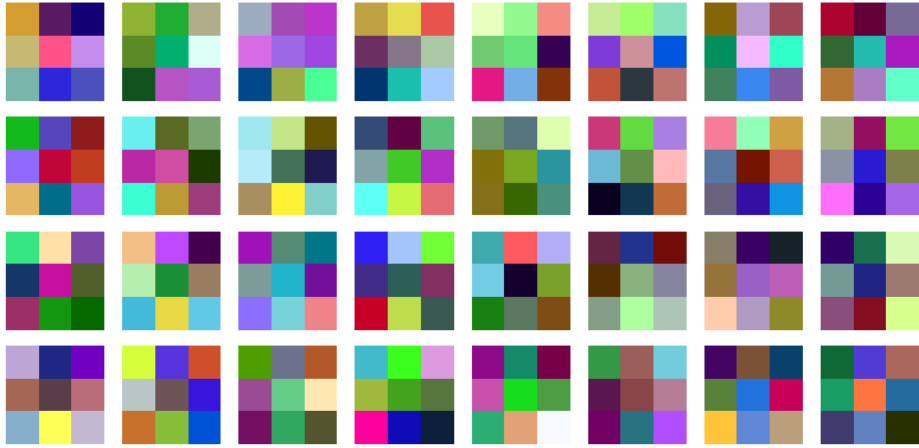


Figure 22: First-layer filters: SV-TF-CNN-5 (top), SV-PT-CNN-5 (bottom).

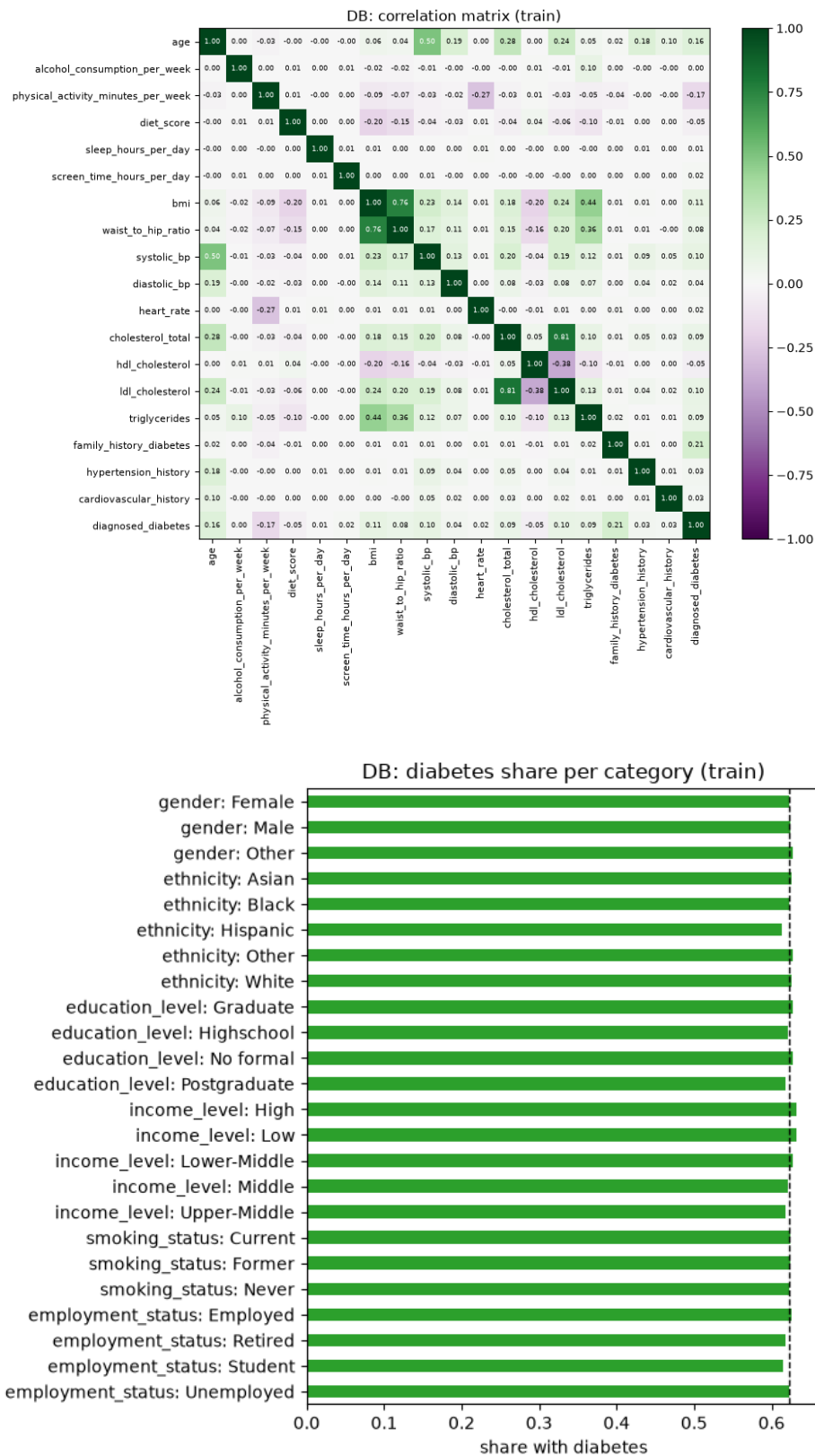


Figure 23: DB: correlation matrix (top) and diabetes share per category (bottom).

A.2 How to run the project

1. Environment (Windows, macOS or Linux; Python 3.10–3.12):

```
python -m venv .venv
.venv\Scripts\activate # Windows
source .venv/bin/activate # macOS / Linux
```

```
pip install -r requirements.txt
```

Regarding GPU support: Apple Silicon picks up `tensorflow-metal` and `mps` automatically; with an NVIDIA GPU, PyTorch uses `cuda` (on Windows the CUDA build must be installed first, see below), whereas TensorFlow can only use an NVIDIA GPU under Linux/WSL2. Absent a GPU, both frameworks fall back to the CPU.

```
pip install torch --index-url https://download.pytorch.org/whl/cu126
```

2. Data. Download from Kaggle and place the files as follows (see `dataset/README.md`):

Folder	Files
<code>dataset/sv/</code>	<code>train_32x32.mat</code> , <code>test_32x32.mat</code>
<code>dataset/gt/</code>	<code>train.p</code> , <code>valid.p</code> , <code>test.p</code> , <code>signname.csv</code>
<code>dataset/db/</code>	<code>train.csv</code> (competition <code>playground-series-s5e12</code> ; join it first)

3. Run the notebooks. Open `sv_svhn.ipynb`, `gt_gtsrb.ipynb` or `db_diabetes.ipynb` from `notebook/` in Jupyter or VS Code and execute all cells, or alternatively run from `notebook/`:

```
jupyter nbconvert --to notebook --execute --inplace db_diabetes.ipynb
```

Every notebook is fully self-contained. The first run generates `dataset/<code>/splits.npz` (the 70/15/15 split), after which it trains all four models and writes out `notebook/results/<run_id>.json`, saves the trained models under `notebook/models/`, and produces the figures in `report/images/`. It is best to start with `db_diabetes`, since it runs the fastest.